



Universidad Internacional de La Rioja
Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Análisis y Visualización de Datos
Masivos/ Visual Analytics and Big Data

Diseño e implementación de una arquitectura de datos masivos para el análisis predictivo y la visualización interactiva

Trabajo fin de estudio presentado por:	Laura Carrasco Mateos
Tipo de trabajo:	Desarrollo de software
Director/a:	Dr. Luis Miguel Fernández-Galván
Fecha:	Mayo de 2026

Resumen

Este Trabajo Fin de Máster tiene como objetivo diseñar e implementar una arquitectura de datos masivos orientada al análisis predictivo y a la visualización interactiva. Para validar la propuesta se ha desarrollado un caso práctico basado en datos abiertos de tráfico urbano de la ciudad de Madrid, utilizando información histórica de intensidad de tráfico correspondiente a los años 2023 y 2024 y datos de ubicación de los puntos de medida publicados por el Ayuntamiento de Madrid. La arquitectura implementada integra fases de ingesta, limpieza, transformación, construcción de un dataset analítico, entrenamiento de modelos predictivos y explotación visual mediante un cuadro de mando interactivo en Power BI.

El proceso de ingesta se ha aplicado sobre 24 ficheros mensuales en formato CSV, con un volumen inicial de 308.458.911 registros. Para construir un prototipo funcional y computacionalmente manejable se seleccionaron los 50 puntos de medida con mayor número de registros válidos. Tras aplicar los procesos de filtrado, conversión de tipos, eliminación de registros erróneos y agregación horaria, se obtuvo un dataset final de 836.228 registros horarios. Posteriormente, se formuló un problema de regresión supervisada cuyo objetivo fue predecir la intensidad horaria del tráfico por punto de medida. Se comparó un modelo base, basado en la intensidad registrada en la misma hora del día anterior, con un modelo Random Forest Regressor.

Los resultados obtenidos muestran que el modelo Random Forest mejora de forma clara al modelo base, reduciendo el MAE de 433,41 a 164,16 vehículos/hora, el RMSE de 840,93 a 311,83 y aumentando el coeficiente R^2 de 0,7569 a 0,9666. Finalmente, los datos limpios, las predicciones, las métricas y las variables explicativas se integraron en un dashboard interactivo compuesto por páginas de calidad de datos, análisis temporal, análisis geográfico y evaluación del modelo predictivo. La arquitectura desarrollada demuestra la viabilidad de integrar datos abiertos, procesamiento analítico, aprendizaje automático y visualización interactiva en una solución modular, reutilizable y orientada a la toma de decisiones.

Palabras clave: datos masivos, análisis predictivo, movilidad urbana, tráfico, Power BI, Random Forest, arquitectura de datos, visualización interactiva.

Palabras clave: datos masivos, análisis predictivo, movilidad urbana, Power BI, visualización interactiva.

Abstract

This Master's Thesis aims to design and implement a big data architecture focused on predictive analytics and interactive visualization. The proposal is validated through a practical case study based on open urban traffic data from the city of Madrid, using historical traffic intensity records from 2023 and 2024 together with the geolocation dataset of traffic measurement points published by the Madrid City Council. The implemented architecture integrates data ingestion, cleaning, transformation, analytical dataset construction, predictive model training and visual exploitation through an interactive dashboard in Power BI.

The ingestion process was applied to 24 monthly CSV files, with an initial volume of 308,458,911 records. In order to build a functional and computationally manageable prototype, the 50 measurement points with the highest number of valid records were selected. After filtering errors, converting data types, removing invalid records and aggregating the data at hourly level, a final clean dataset of 836,228 hourly records was obtained. A supervised regression problem was then defined, with the aim of predicting hourly traffic intensity by measurement point. A baseline model based on the intensity observed at the same hour of the previous day was compared with a Random Forest Regressor.

The results show that the Random Forest model clearly improves the baseline model, reducing the MAE from 433.41 to 164.16 vehicles/hour, the RMSE from 840.93 to 311.83, and increasing the R^2 coefficient from 0.7569 to 0.9666. Finally, the cleaned data, predictions, evaluation metrics and explanatory variables were integrated into an interactive dashboard composed of data quality, temporal analysis, geographical analysis and predictive model evaluation pages. The developed architecture demonstrates the feasibility of integrating open data, analytical processing, machine learning and interactive visualization into a modular and reusable solution aimed at supporting data-driven decision-making.

Keywords: big data, predictive analytics, urban mobility, traffic, Power BI, Random Forest, data architecture, interactive visualization.

Keywords: : Biga Data, predictive analytics, urban mobility, Power BI, interactive visualization.

Índice de contenidos

1.	Introducción	1
1.1.	Motivación	2
1.2.	Planteamiento del trabajo	2
1.3.	Estructura del trabajo	3
2.	Contexto y estado del arte	5
2.1.	Contexto del problema	5
2.2.	Estado del arte	6
2.3.	Conclusiones	8
3.	Objetivos concretos y metodología de trabajo	10
3.1.	Objetivo general	10
3.2.	Objetivos específicos	11
3.3.	Metodología del trabajo	12
4.	Marco normativo	14
5.	Desarrollo específico de la contribución	17
5.1.	“Título 2” del menú de estilos	Error! Bookmark not defined.
5.2.	“Título 2” del menú de estilos	Error! Bookmark not defined.
5.2.1.	“Título 3” del menú de estilos	Error! Bookmark not defined.
6.	Código fuente y datos analizados	29
6.1.	Código fuente	29
6.2.	Datos Analizados	30
7.	Conclusiones	32
8.	Limitaciones y prospectiva	35
8.1.	Limitaciones	35
8.2.	Trabajo futuro	35

Referencias bibliográficas.....	37
---------------------------------	----

Anexo A. Privacidad y protección de datos	39
---	----

Índice de figuras

Figura 1. *Ejemplo de figura realizada para nuestro trabajo.....* **Error! Bookmark not defined.**

Índice de tablas

Tabla 1. *Ejemplo de tabla con sus principales elementos.* **Error! Bookmark not defined.**

1. Introducción

En los últimos años, el incremento en la generación de datos ha impulsado la necesidad de desarrollar soluciones capaces de almacenar, procesar, analizar e interpretar grandes volúmenes de información procedentes de diversas fuentes. Este contexto ha favorecido la adopción de arquitecturas de datos masivos que permiten integrar diferentes fases del ciclo de vida del dato, desde la obtención y transformación de la información hasta su análisis avanzado y visualización final.

Sin embargo, en muchas ocasiones, los datos disponibles se encuentran distribuidos en distintas fuentes, presentan formatos diversos y requieren procesos previos de limpieza, normalización e integración antes de poder ser utilizados de forma analítica. Esto dificulta la extracción de conocimiento útil y limita el aprovechamiento de técnicas predictivas y herramientas de visualización interactiva para apoyar la toma de decisiones.

Este Trabajo Fin de Máster se centra en el diseño e implementación de una arquitectura general de datos masivos orientada al análisis predictivo y la visualización interactiva. Para validar el funcionamiento de la arquitectura propuesta, se aplicará a un caso práctico basado en datos de movilidad urbana.

La elección de la movilidad urbana como caso de validación permite comprobar la capacidad de la arquitectura para integrar datos procedentes de fuentes abiertas, procesarlos de forma estructurada, aplicar técnicas de análisis predictivo y representar los resultados mediante cuadros de mando interactivos. De este modo, el trabajo no se limita a construir una visualización final, sino que aborda todo el proceso técnico necesario para transformar datos en información útil.

La contribución principal del trabajo consiste, por tanto, en una solución tecnológica que integra procesos de ingesta, almacenamiento, transformación, análisis predictivo y visualización interactiva dentro de una arquitectura común. Aunque el caso práctico se sitúa en el ámbito de la movilidad urbana, el enfoque general de la arquitectura permite que sus componentes puedan ser reutilizados o adaptados posteriormente a otros contextos de análisis de datos masivos.

1.1.Motivación

La motivación principal de este TFM surge de la creciente importancia que tiene el análisis de datos masivos en la toma de decisiones dentro de organizaciones, administraciones públicas y entornos empresariales. Actualmente, muchos sectores disponen de grandes cantidades de datos, pero no siempre cuentan con una arquitectura que permita tratarlos de forma integrada, escalable y orientada a la generación de conocimiento.

El ámbito de la movilidad urbana lo he elegido como caso práctico de validación porque representa un escenario realista y complejo para el análisis de datos. Las ciudades generan información relacionada con desplazamientos, tráfico, transporte público, horarios, localización y comportamiento temporal de la demanda. Estos datos pueden resultar útiles para identificar patrones de uso, anticipar necesidades y apoyar decisiones relacionadas con la planificación y gestión de servicios urbanos.

Desde el punto de vista académico, este trabajo se relaciona directamente con los contenidos del Máster en Análisis y Visualización de Datos Masivos, ya que se combinan técnicas de captura y almacenamiento de datos, procesamiento masivo, análisis e interpretación de información, aplicación de técnicas de Inteligencia Artificial y visualización interactiva. Asimismo, el enfoque elegido permite desarrollar una contribución práctica y tecnológica, alineada con un trabajo de tipo desarrollo software

1.2.Planteamiento del trabajo

El problema que aborda este Trabajo Fin de Máster es la dificultad de integrar en una única solución las distintas fases necesarias para trabajar con datos masivos: obtención de datos, almacenamiento, procesamiento, análisis predictivo e interpretación visual de resultados. Aunque existen herramientas especializadas para cada una de estas fases, su uso aislado puede dificultar la construcción de un flujo completo, coherente y reutilizable.

Para dar respuesta a esta necesidad, se propone diseñar e implementar una arquitectura general de datos masivos que permita organizar el proceso completo de análisis. Esta arquitectura se validará mediante un caso práctico centrado en movilidad urbana, permitirá comprobar si la solución propuesta es capaz de trabajar con datos abiertos, procesarlos

correctamente, generar información analítica relevante y ofrecer resultados interpretables mediante un cuadro de mando interactivo. La movilidad urbana no constituye, por tanto, el único alcance conceptual del TFM, sino el escenario elegido para demostrar la utilidad de la arquitectura desarrollada.

El trabajo se enmarca principalmente dentro del Tipo 2 – Desarrollo de software, ya que la aportación principal consiste en una solución tecnológica implementada y evaluada. Aunque la arquitectura se diseña con criterios de modularidad, escalabilidad y reutilización, no se plantea como una metodología nueva, sino como una arquitectura funcional que permite integrar diferentes tecnologías y procesos dentro de un mismo flujo de análisis.

1.3. Estructura del trabajo

El documento se divide en ocho capítulos principales, además de las referencias bibliográficas y los anexos correspondientes.

El primer capítulo introduce el trabajo, presenta la motivación que justifica su realización, describe el planteamiento general de la propuesta y expone la estructura de la memoria.

El segundo capítulo desarrolla el contexto y el estado del arte. En él se analiza el problema de partida y se revisan los principales trabajos, enfoques y soluciones relacionados con arquitecturas Big Data, análisis predictivo aplicado a datos masivos, visualización interactiva y herramientas existentes.

El tercer capítulo presenta los objetivos concretos y la metodología de trabajo. En este apartado se define el objetivo general, se detallan los objetivos específicos y se describe el proceso metodológico seguido para el desarrollo del TFM, combinando un enfoque propio de proyectos de análisis de datos con un desarrollo iterativo e incremental.

El cuarto capítulo aborda el marco normativo aplicable. Se revisarán las consideraciones relacionadas con la protección de datos personales, la reutilización de información procedente de fuentes abiertas y las implicaciones normativas del tratamiento de datos en el ámbito seleccionado.

El quinto capítulo recoge el desarrollo específico de la contribución. En él se describirá la arquitectura propuesta, sus componentes principales, las tecnologías seleccionadas, el flujo

de procesamiento de datos, los modelos analíticos utilizados y el diseño del cuadro de mando interactivo.

El sexto capítulo estará dedicado al código fuente y a los datos analizados. Se documentarán las fuentes de datos empleadas, su estructura, las transformaciones realizadas y la organización del código desarrollado durante el proyecto.

El séptimo capítulo expondrá las conclusiones del trabajo, relacionando los resultados obtenidos con los objetivos planteados inicialmente y valorando el grado de cumplimiento de cada uno de ellos.

Finalmente, el octavo capítulo presentará las limitaciones encontradas durante el desarrollo del TFM y las posibles líneas de trabajo futuro, tanto desde el punto de vista técnico como desde la perspectiva de ampliación de la arquitectura a otros dominios de aplicación.

2. Contexto y estado del arte

2.1.Contexto del problema

El crecimiento continuo de la generación de datos ha transformado la forma en que las organizaciones, administraciones públicas y empresas abordan el análisis de información. En entornos digitales, los datos proceden de fuentes cada vez más diversas, como sensores, aplicaciones móviles, sistemas transaccionales, plataformas web, redes de transporte, dispositivos IoT o repositorios públicos. Esta realidad plantea nuevos retos relacionados con el volumen, la variedad, la velocidad y la calidad de los datos, que no siempre pueden abordarse mediante soluciones tradicionales de almacenamiento y análisis.

El concepto de Big Data se utiliza para describir grandes cantidades de datos generadas en un entorno digitalizado, conectado y sensorizado. Las arquitecturas de datos masivos permiten organizar el ciclo de vida del dato desde su origen hasta su uso final. Una arquitectura bien definida facilita que los datos procedentes de distintas fuentes puedan capturarse, limpiarse, almacenarse, procesarse y analizarse de forma coherente. Además, cuando se incorporan técnicas de aprendizaje automático y herramientas de visualización interactiva, estas arquitecturas pueden ayudar a transformar datos heterogéneos en información útil para apoyar la toma de decisiones.

Uno de los ámbitos donde estas necesidades se observan con claridad es la movilidad urbana. Las ciudades generan grandes cantidades de datos relacionados con tráfico, transporte público, desplazamientos, infraestructura viaria, bicicletas compartidas, sensores urbanos y patrones temporales de uso.

En el caso de Madrid, existen fuentes abiertas que pueden servir para validar este tipo de arquitectura. El portal de datos abiertos del Ayuntamiento de Madrid publica datos históricos de tráfico desde 2013, con actualización mensual, y también ofrece datos de tráfico en tiempo real actualizados aproximadamente cada cinco minutos. Además, el portal Open Data de EMT Madrid ofrece conjuntos de datos y una API REST documentada relacionada con movilidad, autobuses y BiciMAD. Estas fuentes permiten plantear un caso práctico basado en datos abiertos, adecuado para analizar patrones temporales, comportamiento de la demanda y posibles predicciones de uso o intensidad.

A pesar de la disponibilidad de datos, el problema no queda resuelto únicamente con su publicación. Los datos abiertos suelen encontrarse en formatos diversos, con distintos niveles de granularidad, periodicidad y calidad. Por ello, para explotarlos de forma analítica es necesario diseñar procesos de extracción, transformación, almacenamiento y modelado que permitan generar información fiable. La contribución de este trabajo se sitúa precisamente en ese punto: diseñar e implementar una arquitectura general que integre esas fases y validarla mediante un caso práctico de movilidad urbana.

2.2. Estado del arte

2.2.1. Arquitecturas Big Data para el análisis de datos masivos

Las arquitecturas Big Data surgen como respuesta a la necesidad de gestionar grandes volúmenes de datos procedentes de fuentes heterogéneas. Estas arquitecturas permiten organizar el ciclo de vida del dato desde su obtención hasta su explotación final, incorporando fases de almacenamiento, procesamiento, transformación, análisis y visualización. A diferencia de los sistemas tradicionales, las soluciones Big Data deben ser escalables y capaces de trabajar con datos de distinta naturaleza, tanto estructurados como semiestructurados o no estructurados.

Entre los enfoques más habituales se encuentran los almacenes de datos, los data lakes y las arquitecturas por capas. Los almacenes de datos resultan adecuados para información estructurada y orientada a consulta, mientras que los data lakes permiten conservar datos en su formato original para tratarlos posteriormente según las necesidades analíticas. También son relevantes arquitecturas como Lambda, que combina procesamiento por lotes y en tiempo real, o Kappa, más centrada en el tratamiento continuo de flujos de datos.

En este trabajo se toma como referencia una arquitectura modular, compuesta por varias capas: obtención de datos, almacenamiento, procesamiento, análisis predictivo y visualización interactiva. Esta estructura permite separar responsabilidades, facilitar la escalabilidad y adaptar la solución a distintos dominios. Aunque la validación se realizará mediante un caso práctico de movilidad urbana, la arquitectura se plantea con carácter general y reutilizable.

2.2.2. Análisis predictivo aplicado a la movilidad urbana

La movilidad urbana constituye un ámbito adecuado para aplicar técnicas de análisis predictivo, ya que genera grandes cantidades de datos temporales y geográficos. Estos datos

pueden proceder de sensores de tráfico, transporte público, bicicletas compartidas, registros de desplazamientos o datos abiertos publicados por administraciones públicas. Su análisis permite identificar patrones de comportamiento, anticipar situaciones de alta demanda y apoyar la planificación de servicios urbanos.

A lo largo del tiempo se han utilizado diferentes técnicas para la predicción de tráfico y demanda de transporte. Entre los métodos más habituales se encuentran los modelos estadísticos de series temporales, los algoritmos de aprendizaje automático supervisado y, en trabajos más avanzados, técnicas de aprendizaje profundo. Modelos como árboles de decisión, Random Forest, Gradient Boosting o redes neuronales se emplean para estimar variables como la intensidad del tráfico, el número de desplazamientos o la demanda por zonas y franjas horarias.

Para este TFM, el análisis predictivo se plantea como una fase integrada dentro de la arquitectura general. No se busca únicamente entrenar un modelo aislado, sino incorporarlo dentro de un flujo completo que incluya preparación de datos, entrenamiento, evaluación y posterior visualización de los resultados.

2.2.3. Visualización interactiva de datos de movilidad

La visualización interactiva es una parte esencial de los proyectos de análisis de datos, ya que permite interpretar de forma clara los resultados obtenidos. En el caso de la movilidad urbana, los datos presentan una fuerte dimensión temporal y geográfica, por lo que resulta especialmente útil emplear gráficos de evolución, mapas, indicadores clave y filtros interactivos.

Los cuadros de mando permiten integrar en una misma interfaz diferentes visualizaciones y segmentadores, facilitando que el usuario explore la información desde varias perspectivas. En lugar de presentar únicamente tablas o resultados numéricos, un dashboard permite analizar patrones horarios, comparar zonas, observar tendencias y detectar comportamientos relevantes.

En este trabajo se propone utilizar Power BI como herramienta de visualización interactiva. Esta elección permite construir informes dinámicos, integrar indicadores derivados del análisis predictivo y facilitar la interpretación de los datos por parte de usuarios no técnicos. De este

modo, la visualización se convierte en la capa final de la arquitectura, donde los datos procesados y los resultados analíticos se transforman en información comprensible.

2.2.4. Herramientas y soluciones existentes

Existen diversas herramientas y plataformas que abordan parcialmente el problema tratado en este TFM. Por un lado, los portales de datos abiertos, como los de administraciones públicas, ofrecen conjuntos de datos relacionados con tráfico, transporte o movilidad urbana. Sin embargo, estos datos suelen estar dispersos, en diferentes formatos y requieren procesos previos de limpieza e integración.

Por otro lado, tecnologías como Apache Spark permiten procesar grandes volúmenes de datos y aplicar transformaciones de forma escalable. También existen herramientas de inteligencia de negocio, como Power BI o Tableau, que facilitan la creación de cuadros de mando interactivos. Sin embargo, estas herramientas no resuelven por sí solas todo el ciclo de vida del dato, especialmente cuando es necesario integrar múltiples fuentes, aplicar modelos predictivos y organizar el flujo completo de procesamiento.

El análisis de estas soluciones muestra que existen herramientas útiles para cada fase del proceso, pero no siempre se integran en una arquitectura completa y reutilizable. Por ello, este TFM propone una arquitectura general que combine obtención, almacenamiento, procesamiento, análisis predictivo y visualización interactiva dentro de un mismo flujo de trabajo, validándola mediante un caso práctico de movilidad urbana.

2.3. Conclusiones

La revisión del estado del arte permite concluir que las arquitecturas Big Data son necesarias para trabajar con datos masivos procedentes de fuentes distintas. Estas arquitecturas permiten estructurar el proceso completo de tratamiento de datos, desde la obtención inicial hasta la generación de resultados analíticos y visuales.

Es por ello que el análisis predictivo aplicado a la movilidad urbana constituye un campo relevante, ya que permite anticipar patrones de tráfico, demanda de transporte o comportamiento de los desplazamientos. La información que se obtiene muestra que existen

distintas técnicas aplicables, desde modelos estadísticos hasta algoritmos de aprendizaje automático y aprendizaje profundo.

También se observa que la visualización interactiva resulta fundamental para interpretar los resultados y facilitar la toma de decisiones. En este sentido, los cuadros de mando permiten representar patrones temporales, espaciales y predictivos de forma accesible para usuarios no técnicos.

Finalmente, aunque existen portales de datos abiertos, herramientas de procesamiento y plataformas de visualización, muchas soluciones abordan estas fases de forma separada. Por ello, el principal hueco identificado es la necesidad de integrar todos estos componentes en una arquitectura común, modular y reutilizable. Esta conclusión justifica la propuesta de este TFM: diseñar e implementar una arquitectura general de datos masivos para análisis predictivo y visualización interactiva, validada mediante un caso práctico de movilidad urbana.

3. Objetivos concretos y metodología de trabajo

Este apartado explica los objetivos concretos del Trabajo Fin de Máster y la metodología que se seguirá para su desarrollo. A partir del contexto y del estado del arte expuestos anteriormente, se define una contribución centrada en el diseño e implementación de una arquitectura general de datos masivos, orientada al análisis predictivo y a la visualización interactiva.

El trabajo se enmarca principalmente dentro del Tipo 2 – Desarrollo de software, ya que la contribución principal consiste en la implementación de una solución tecnológica que integra distintos componentes del ciclo de vida del dato: obtención, almacenamiento, procesamiento, análisis predictivo y visualización interactiva. No se plantea como una metodología nueva de carácter general, sino como una arquitectura funcional que puede ser aplicada y adaptada a distintos contextos de análisis de datos masivos.

3.1. Objetivo general

El objetivo general de este proyecto es el diseño y la implementación de una arquitectura general de datos masivos orientada al análisis predictivo y la visualización interactiva, integrando procesos de obtención, almacenamiento, transformación, modelado analítico y representación visual de la información, y validando su funcionamiento mediante un caso práctico aplicado a datos abiertos de movilidad urbana.

Con este objetivo se pretende construir una solución que permita trabajar con datos procedentes de fuentes distintas, prepararlos para su análisis, aplicar técnicas de aprendizaje automático y representar los resultados mediante cuadros de mando interactivos. La movilidad urbana se utiliza como escenario de validación porque permite trabajar con datos temporales, geográficos y operativos, lo que facilita comprobar la capacidad de la arquitectura para integrar, procesar y analizar información de distinta naturaleza.

La finalidad última del trabajo no es únicamente desarrollar un cuadro de mando ni entrenar un modelo predictivo aislado, sino integrar ambos elementos dentro de una arquitectura completa que permita transformar datos en conocimiento útil. De esta forma, la solución

propuesta podrá servir como base para apoyar la toma de decisiones en escenarios donde sea necesario interpretar grandes volúmenes de datos y anticipar comportamientos futuros.

3.2. Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

1. Identificar y analizar los requisitos funcionales y técnicos de una arquitectura de datos masivos orientada al análisis predictivo y a la visualización interactiva.
2. Seleccionar fuentes de datos abiertas adecuadas para validar la arquitectura propuesta, utilizando como caso práctico conjuntos de datos relacionados con la movilidad urbana.
3. Diseñar una arquitectura modular que integre las fases de obtención, almacenamiento, procesamiento, análisis predictivo y visualización de datos masivos.
4. Implementar procesos de extracción, transformación y carga de datos que permitan limpiar, normalizar e integrar información procedente de distintas fuentes.
5. Construir un modelo de datos que facilite el análisis de la información y permita relacionar variables temporales, geográficas y operativas.
6. Aplicar técnicas de análisis exploratorio para identificar patrones, tendencias y relaciones relevantes dentro del caso práctico seleccionado.
7. Desarrollar modelos predictivos que permitan estimar comportamientos futuros a partir de los datos históricos disponibles.
8. Diseñar e implementar un cuadro de mando interactivo que permita visualizar los resultados del análisis y facilitar su interpretación por parte de usuarios no técnicos.
9. Evaluar la arquitectura propuesta mediante métricas de rendimiento, calidad de los resultados predictivos y utilidad de las visualizaciones generadas.
10. Valorar la reutilización de la arquitectura en otros escenarios de análisis de datos masivos, identificando sus limitaciones y posibles líneas de mejora.

3.3. Metodología del trabajo

La metodología seguida en este Trabajo Fin de Máster combina un enfoque estructurado propio de proyectos de análisis de datos con una organización iterativa del desarrollo software. Para ello, se toma como referencia el modelo CRISP-DM (Cross Industry Standard Process for Data Mining), ya que permite organizar el trabajo en fases relacionadas con la comprensión del problema, comprensión de los datos, preparación, modelado, evaluación y despliegue de resultados. De forma complementaria, se utilizará un enfoque ágil basado en Kanban, que permitirá gestionar las tareas de forma incremental y controlar el avance del proyecto mediante actividades pendientes, en desarrollo, en revisión y finalizadas.

El desarrollo del trabajo se organizará en las siguientes fases:

1. Definición del problema y alcance de la arquitectura.

En esta fase se delimita el objetivo del TFM, diferenciando entre la arquitectura general propuesta y el caso práctico utilizado para validarla. La arquitectura se plantea como una solución modular para el análisis predictivo y la visualización interactiva de datos masivos, mientras que la movilidad urbana se utilizará como escenario de prueba.

2. Selección de fuentes de datos.

Se identificarán y seleccionarán fuentes de datos abiertas relacionadas con movilidad urbana, priorizando conjuntos de datos públicos, reutilizables y sin información personal identificable. Entre las posibles fuentes se contemplan datos abiertos del Ayuntamiento de Madrid y del portal Open Data de EMT Madrid.

3. Diseño de la arquitectura.

Se definirá una arquitectura compuesta por módulos de obtención, almacenamiento, procesamiento, análisis predictivo y visualización interactiva. Esta estructura permitirá separar responsabilidades y facilitar la reutilización de la solución en otros contextos de datos masivos.

4. Obtención, almacenamiento y transformación de datos.

Se implementarán procesos de extracción, transformación y carga para limpiar, normalizar e integrar los datos seleccionados. Se tratarán valores nulos, formatos de fecha, variables

geográficas y posibles inconsistencias. Para el almacenamiento se valorará el uso de ficheros estructurados y una base de datos relacional como PostgreSQL.

5. Análisis exploratorio y modelado predictivo.

Se realizará un análisis exploratorio para identificar patrones temporales, geográficos y operativos. Posteriormente, se aplicarán técnicas de aprendizaje automático para estimar comportamientos futuros, como la intensidad del tráfico o la demanda de movilidad.

6. Visualización interactiva de resultados.

Los resultados obtenidos se representarán mediante un cuadro de mando en Power BI, incorporando indicadores, gráficos temporales, mapas, filtros y comparativas entre valores reales y predichos. Esta fase permitirá transformar los resultados analíticos en información comprensible para usuarios no técnicos.

7. Evaluación de la arquitectura.

La solución se evaluará desde tres perspectivas: técnica, analizando el correcto funcionamiento del flujo de datos; analítica, valorando la calidad de los modelos predictivos; y visual, revisando la claridad y utilidad del cuadro de mando desarrollado.

Esta metodología permite desarrollar el TFM de forma ordenada, manteniendo la coherencia entre los objetivos, la arquitectura propuesta, el caso práctico de validación y los resultados esperados.

4. Marco normativo

Este TFM se basa en el diseño e implementación de una arquitectura de datos masivos validada mediante un caso práctico de movilidad urbana. Por ello, el marco normativo aplicable se centra principalmente en la protección de datos personales, la reutilización de información del sector público y la normativa relacionada con movilidad.

4.1. Protección de datos personales

En relación con la protección de datos, las normas principales son el Reglamento General de Protección de Datos y la Ley Orgánica 3/2018, que regula en España el tratamiento de datos personales y adapta el ordenamiento jurídico nacional al RGPD.

En este TFM se priorizará el uso de datos abiertos, agregados o anonimizados, procedentes de fuentes institucionales. No se prevé tratar datos personales identificados o identificables, como nombres, documentos de identidad, matrículas, identificadores de dispositivos o trayectorias individuales.

La Agencia Española de Protección de Datos indica que el RGPD no afecta al tratamiento de información anónima cuando esta no guarda relación con una persona física identificada o identificable, o cuando los datos se han anonimizado de forma que la persona no pueda ser identificada.

Por tanto, si los conjuntos de datos utilizados son agregados o están suficientemente anonimizados, el tratamiento principal quedaría fuera del ámbito de aplicación del RGPD. Aun así, se aplicarán buenas prácticas como la minimización de datos, el uso exclusivo para fines académicos y la documentación de las transformaciones realizadas.

4.2. Reutilización de información del sector público

Al utilizar datos abiertos publicados por administraciones públicas, también resulta aplicable la Ley 37/2007, sobre reutilización de la información del sector público. Esta norma regula el uso de documentos y datos elaborados o custodiados por administraciones y organismos públicos, tanto con fines comerciales como no comerciales.

En este trabajo, la reutilización de datos tendrá una finalidad académica. Por ello, se respetarán las condiciones de uso de las fuentes seleccionadas, citando siempre el origen de los datos, indicando la fecha de descarga o actualización cuando esté disponible y evitando alterar el sentido de la información original.

4.3. Normativa relacionada con movilidad

Dado que el caso práctico se aplicará al ámbito de la movilidad urbana, también se tendrá en cuenta la Ley 9/2025, de Movilidad Sostenible. Esta norma establece un marco general para promover un sistema de movilidad sostenible, seguro, accesible, digitalizado y orientado a mejorar la planificación y gestión del transporte.

El TFM no intervendrá sobre sistemas reales de transporte ni tomará decisiones automatizadas con efectos sobre personas. Su finalidad será únicamente académica y analítica. No obstante, el marco actual de movilidad refuerza la importancia de utilizar datos para mejorar el análisis, la planificación y la toma de decisiones en este ámbito.

4.4. Consideraciones éticas y de seguridad

Aunque el trabajo se base en datos abiertos, es necesario aplicar criterios éticos en el tratamiento de la información. El hecho de que un conjunto de datos sea público no implica que pueda utilizarse sin atender a su contexto, condiciones de reutilización o posibles riesgos de reidentificación.

Por ello, durante el desarrollo del TFM se seguirán las siguientes medidas:

- Se utilizarán preferentemente datos agregados, anonimizados o sin identificadores personales.
- No se intentará identificar a personas a partir de datos anonimizados.
- No se combinarán fuentes con la finalidad de reconstruir trayectorias individuales.
- Se citará correctamente la fuente de los datos utilizados.
- Se documentarán las transformaciones aplicadas sobre los conjuntos de datos.
- Se emplearán los datos únicamente con finalidad académica.

- Se evitará publicar información que pueda afectar a la privacidad, seguridad o derechos de terceros.

Estas medidas permiten reducir los riesgos asociados al tratamiento de información de movilidad y garantizan que el desarrollo de la arquitectura se realice de forma responsable.

4.5. Aplicación al TFM

En conclusión, el TFM se desarrollará bajo un enfoque preventivo y responsable. Se utilizarán preferentemente datos abiertos, agregados o anonimizados, evitando cualquier tratamiento de información personal identificable. Además, se respetarán las condiciones de reutilización de los datos públicos y se documentará el origen, formato y transformaciones aplicadas sobre los conjuntos de datos utilizados.

Este marco normativo permite justificar el uso de datos abiertos de movilidad urbana dentro de una arquitectura de análisis predictivo y visualización interactiva, reduciendo riesgos legales y garantizando un tratamiento responsable de la información.

5. Desarrollo específico de la contribución

Este apartado contiene el núcleo principal del Trabajo Fin de Máster y describe la contribución desarrollada. Lo que se propone es una arquitectura funcional de datos masivos orientada al análisis predictivo y a su visualización interactiva. Aunque esta arquitectura se ha diseñado con un carácter modular y reutilizable por el contenido, se ejemplifica con un caso práctico: la predicción de la intensidad horaria del tráfico urbano en la ciudad de Madrid a partir de datos abiertos del Ayuntamiento.

El desarrollo no solo es una descripción conceptual del tema elegido, sino que incluye una implementación técnica completa. Se han descargado y organizado datos reales de distintos años, luego se ha elaborado un proceso de ingesta y limpieza mediante Python, se ha generado un dataset analítico, se ha evaluado un modelo predictivo y para finalizar se ha diseñado un dashboard interactivo en Power BI. De esta forma, el TFM responde al tipo de trabajo de desarrollo de software, ya que tiene como aportación principal una solución tecnológica implementada, evaluada y documentada.

5.1. Descripción general de la arquitectura propuesta

La arquitectura propuesta en el trabajo se estructura en seis capas principales: adquisición de datos, ingesta, procesamiento y limpieza, construcción del dataset analítico, modelado predictivo y visualización interactiva. Cada capa tiene una responsabilidad concreta dentro del ciclo de vida del dato y se conecta con la siguiente mediante ficheros intermedios. Esta organización permite mantener la trazabilidad del proceso, identificar posibles errores y reutilizar componentes en otros escenarios.

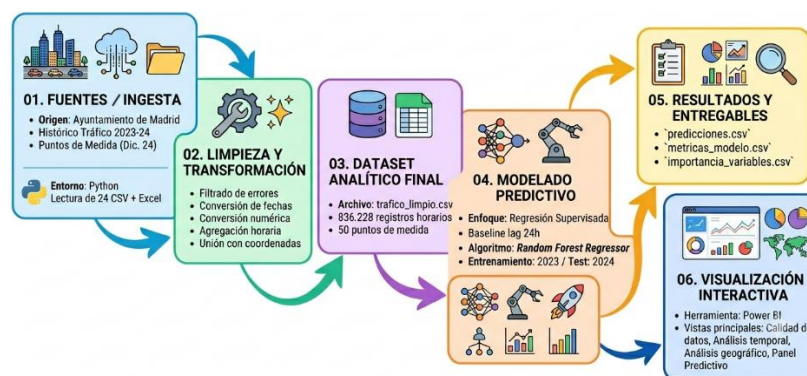


Figura 1. Arquitectura general propuesta para el análisis predictivo y la visualización interactiva. Elaboración propia

Capa	Función	Tecnologías
Adquisición de datos	Localización y descarga de fuentes abiertas relevantes para el caso de estudio.	Portal de datos abiertos: Ayuntamiento de Madrid
Ingesta	Lectura automatizada de ficheros mensuales de dos años y control del volumen de datos.	Python, pandas y lectura por bloques
Procesamiento y limpieza	Filtrado de errores, conversión de tipos, eliminación de registros inválidos y agregación horaria.	Python y pandas
Dataset analítico	Generación de una tabla final con variables temporales, operativas y geográficas.	CSV procesados
Modelado predictivo	Entrenamiento y evaluación de modelos de regresión para predecir la intensidad horaria.	Scikit-learn
Visualización	Construcción de un dashboard interactivo con análisis temporal, geográfico y predictivo	Power BI

Tabla 1. Componentes principales de la arquitectura propuesta. Elaboración propia.

5.2. Instanciación de la arquitectura en el caso de la movilidad urbana

Para validar la arquitectura se ha elegido un caso práctico de movilidad urbana basado en el tráfico de Madrid. Esta elección permite trabajar con datos abiertos, reales, temporales, de gran volumen y con referencias geográficas. El problema predictivo consiste en estimar la intensidad horaria del tráfico para cada punto de medida seleccionado. La unidad de análisis final es, por tanto, la combinación de un punto de medida y una hora concreta.

La decisión de trabajar con la intensidad horaria permite reducir la complejidad del problema y construir una variable objetivo que sea clara. En los datos originales, la información aparece con una granularidad inferior a una hora. Sin embargo, para el análisis predictivo y la visualización se ha considerado más adecuado agregar los registros a nivel horario. Esta agregación permite capturar patrones diarios y semanales de forma interpretable, reduce ruido puntual y facilita la comparación entre periodos.



Figura 2. Arquitectura aplicada al caso práctico de predicción de intensidad del tráfico urbano de Madrid. Elaboración propia.

5.3. Tecnologías seleccionadas y justificación

La implementación se ha realizado con herramientas accesibles, reproducibles y adecuadas para un entorno académico. Se ha utilizado Python como lenguaje principal de desarrollo por su amplio conocimiento en ciencia de datos, su variedad de bibliotecas y su facilidad para automatizar procesos de ingesta, limpieza y modelado. La biblioteca *pandas* se ha empleado para la manipulación de los datos estructurados y *scikit-learn* para el entrenamiento y la evaluación de los modelos predictivos.

Aunque en el estado del arte se revisan tecnologías de procesamiento como Apache Spark, en la implementación final se optó por un procesamiento por bloques con *pandas*, como otra opción. Esta decisión responde a las condiciones del trabajo: desarrollarlo en un entorno local, con recursos propios y se busca una arquitectura funcional antes que desplegar una infraestructura distribuida completa. Para evitar limitaciones en la memoria, los ficheros se

han procesado por fragmentos de 500.000 registros. Esto permite trabajar con ficheros que sean de gran tamaño sin cargar todo el volumen de datos al mismo tiempo.

Power BI se ha seleccionado como herramienta de visualización por la facilidad con la que se conectan ficheros CSV, se construyen informes interactivos y se combinan gráficos temporales, mapas, tablas y tarjetas de indicadores. Esta herramienta permite que la capa final de la arquitectura sea comprensible para usuarios sin una base técnica, facilitando la interpretación de los resultados del modelo predictivo y de los patrones presentes en los datos.

Tecnología	Uso en el proyecto	Justificación
Python	Automatización del pipeline de ingesta, limpieza, transformación y modelado.	Lenguaje flexible y ampliamente utilizado en el análisis de datos.
pandas	Lectura por bloques, limpieza, agregación horaria y generación de datasets.	Permite trabajar con datos tabulares y exportar resultados reproducibles.
scikit-learn	Entrenamiento y evaluación de modelos predictivos.	Incluye algoritmos de aprendizaje automático y métricas de evaluación.
Power BI	Diseño del dashboard interactivo.	Permite visualizar datos, aplicar filtros y comunicar resultados.
CSV	Intercambio entre capas de las arquitectura.	Formato simple, trazable y compatible con Python y Power BI.

Tabla 2. Tecnologías seleccionadas para la implementación. Elaboración propia.

5.4. Fuentes de datos utilizadas

El caso práctico se ha construido a partir de dos fuentes principales procedentes del portal de datos abiertos del Ayuntamiento de Madrid. La primera fuente corresponde al histórico de datos del tráfico por meses, que recoge registros mensuales asociados a los puntos de medida de la ciudad. La segunda fuente escogida corresponde a la ubicación de los puntos de medida, que permite asociar cada identificador con información geográfica como la latitud y la longitud.

Fuentes	Formato	Periodo utilizado	Uso en el TFM
Histórico de datos del tráfico desde 2013.	CSV mensual	2023 y 2024	Registros de intensidad, ocupación, carga, velocidad media, error y periodo de integración.
Ubicación de los puntos de medida del tráfico.	Excel	Diciembre de 2024	Identificador del punto, nombre, distrito, coordenadas, longitud y latitud.

Tabla 3. Fuentes de datos utilizadas en el caso práctico. Elaboración propia.

Los ficheros históricos contienen los campos id, fecha, tipo_elem, intensidad, ocupacion, carga, vmed, error y periodo_integracion. El campo id identifica el punto de medida y se utiliza como foreign key con la tabla geográfica. El campo fecha permite ordenar temporalmente los registros y construir variables derivadas. El campo intensidad constituye la base de la variable objetivo del modelo predictivo. El campo error permite descartar observaciones que no son válidas antes de construir el dataset analítico.

Campos principales de los ficheros históricos de tráfico:

- Id → Identificador del punto de medida.
- Fecha → Fecha y hora del registro original.
- tipo_elem → Tipo de elemento de medición.
- Intensidad → Número de vehículos registrados en el periodo de integración.
- Ocupación → Porcentaje o medida de ocupación asociada al punto.

- Carga → Indicador operativo de carga del tráfico.
- Vmed → Velocidad media registrada.
- Error → Indicador de validez del registro.
- periodo_integracion → Ventana temporal original del registro.

Campos principales del fichero de puntos de medida:

- tipo_elem → Tipo de elemento.
- Distrito → Distrito asociado al punto de medida.
- Id → Identificador común con el histórico de tráfico.
- cod_cent → Código del centro o sistema asociado.
- Nombre → Nombre descriptivo del punto de medida.
- utm_x / utm_y → Coordenadas UTM.
- longitud / latitud → Coordenadas geográficas utilizadas en Power BI.

5.5. Proceso de ingesta de datos

El proceso de ingesta se ha implementado con el script 01_ingesta_limpieza.py. Este script trata de localizar automáticamente los ficheros CSV situados en la carpeta datos/originales/historico_trafico y el fichero Excel situado en datos/originales/puntos_medida. La lectura de los datos históricos se ha realizado por bloques de 500.000 registros, con el fin de reducir el consumo de memoria y permitir el procesamiento de ficheros mensuales de gran tamaño en un equipo local de una mejor forma.

En total se han procesado 24 ficheros mensuales, que corresponden a los doce meses de 2023 y los doce meses de 2024. El volumen original leído ha sido de 308.458.911 registros. Dado que el objetivo de la implementación era validar una arquitectura funcional y no procesar de forma concreta todos los sensores existentes, se han seleccionado los 50 puntos de medida con mayor número de registros válidos. Esta selección ha permitido construir un prototipo que se manejase mejor y, al mismo tiempo, suficientemente representativo para demostrar el funcionamiento de la arquitectura.

Resumen cuantitativo del proceso de ingesta y limpieza:

- Ficheros históricos procesados → 24

- Periodo temporal → Enero de 2023 - diciembre de 2024
- Registros originales leídos → 308.458.911
- Registros válidos tras limpieza inicial → 3.339.435
- Registros horarios generados → 836.228
- Puntos de medida seleccionados → 50
- Periodo final del dataset → 01-01-2023 00:00:00 a 31-12-2024 23:00:00

5.6. Limpieza, transformación y agregación horaria

La fase de limpieza tuvo como finalidad convertir los registros originales en un dataset coherente y adecuado para el análisis predictivo. En primer lugar, se filtraron los registros cuyo campo error no correspondía a un valor válido. A continuación, se pudo convertir el campo fecha a formato fecha-hora y se transformaron las variables numéricas para evitar errores derivados de formatos incorrectos. También se eliminaron registros sin fecha, sin identificador de punto de medida o con intensidad negativa.

Una vez que se aplicaron los filtros de calidad, se realizó una agregación horaria. Para cada combinación de punto de medida y hora se calculó la suma de la intensidad y la media de variables como ocupación, carga y velocidad media. Además, se conservó el número de registros originales que contribuían a cada observación horaria. Esta información resulta útil para valorar la completitud de cada agregación.

Problema detectado	Tratamiento aplicado
Datos repartidos en ficheros mensuales	Lectura automática de los 24 CSV desde la carpeta de datos originales.
Volumen elevado de registros	Procesamiento por bloques de 500.000 filas para evitar problemas de memoria.
Registros marcados con error	Filtrado de observaciones cuyo campo error no era válido.
Campos temporales en formato texto	Conversión del campo fecha a formato fecha-hora.

Variables numéricas con posibles inconsistencias	Conversión de intensidad, ocupación, carga, velocidad media y periodo de integración a formato numérico.
Registros sin fecha o sin punto de medida	Eliminación de observaciones incompletas.
Granularidad inferior a una hora	Agregación por id de punto de medida y hora.
Ausencia de dimensión geográfica en el histórico	Unión con la tabla de puntos de medida mediante el campo id.

Tabla 4. Principales tratamientos aplicados durante la limpieza. Elaboración propia.

5.7.Limpieza, transformación y agregación horaria

El resultado del proceso anterior es un dataset horario en el que cada fila representa la intensidad registrada por un punto de medida en una franja horaria concreta. Este dataset incorpora variables temporales, variables operativas y variables geográficas. Las variables temporales permiten analizar patrones por año, mes, día, hora, día de la semana y fin de semana. Las variables operativas describen el comportamiento del tráfico en cada punto de medida. Las variables geográficas permiten representar espacialmente los resultados en Power BI.

Grupo de variables	Variables incluidas
Identificación	id, nombre, distrito
Tiempo	fecha_hora, año, mes, dia, hora, dia_semana, es_fin_semana
Tráfico	intensidad_horaria, ocupacion_media, carga_media, velocidad_media, registros_15min
Geografía	latitud, longitud

Modelado	variables retardadas y medias móviles generadas posteriormente
----------	--

Tabla 5. Estructura del dataset analítico final. Elaboración propia.

5.8. Modelado predictivo de intensidad del tráfico

El análisis predictivo se ha planteado como un problema de regresión supervisada. La variable objetivo es la `intensidad_horaria`, definida como la suma de la intensidad registrada en una hora concreta para un punto de medida determinado. Esta variable permite estimar el volumen de tráfico esperado en un punto y su franja horaria, lo que resulta útil para anticipar patrones de congestión y apoyar la toma de decisiones en movilidad urbana.

Para construir el modelo se han generado variables explicativas temporales e históricas. Entre las variables temporales se incluyeron el año, el mes, el día, la hora, el día de la semana y un indicador de fin de semana. Entre las variables históricas se incorporaron la intensidad de la hora anterior, la intensidad de la misma hora del día anterior, una media móvil de 3 horas y una media móvil de 24 horas. Estas variables permiten capturar la continuidad temporal y la periodicidad diaria del tráfico.

- Tipo de problema → Regresión supervisada
- Variable objetivo → `intensidad_horaria`
- Unidad de análisis → Punto de medida y hora
- Modelo base → Intensidad registrada en la misma hora del día anterior
- Modelo de aprendizaje automático → Random Forest Regressor
- Entrenamiento → Datos del año 2023
- Prueba → Datos del año 2024
- Métricas → MAE, RMSE y R^2

5.9. Evaluación del modelo predictivo

La evaluación se ha realizado comparando el modelo Random Forest con un modelo base. El modelo base utiliza como predicción la intensidad registrada en la misma hora del día anterior, por lo que constituye una referencia sencilla pero razonable en un problema con periodicidad

Laura Carrasco Mateos

Diseño e implementación de una arquitectura de datos masivos para el análisis predictivo y la visualización interactiva

diaria. La comparación con este modelo permite comprobar si el algoritmo de aprendizaje automático aporta una mejora real frente a una regla temporal simple.

La división entre entrenamiento y prueba se ha realizado de forma temporal. Los datos del año 2023 se han utilizado para entrenar los modelos, mientras que los datos del año 2024 se reservaron para evaluar su rendimiento. Esta estrategia es más adecuada que una división aleatoria, ya que respeta el orden temporal de los datos y permite simular un escenario en el que se entrena con información histórica para predecir un periodo posterior.

Modelo	MAE	RMSE	R ²
Baseline lag 24h	433,41	840,93	0,7569
Random Forest Regressor	164,16	311,83	0,9666

Tabla 6. Métricas de evaluación del modelo predictivo. Elaboración propia.

Los resultados muestran una mejora clara del modelo Random Forest respecto al modelo base. El MAE se reduce de 433,41 a 164,16 vehículos/hora y el RMSE disminuye de 840,93 a 311,83. Además, el coeficiente R² aumenta de 0,7569 a 0,9666, lo que indica que el modelo es capaz de explicar una proporción muy elevada de la variabilidad observada en la intensidad horaria del tráfico. Esta mejora evidencia que las variables temporales, operativas e históricas incorporadas al modelo aportan información relevante para la predicción.

Variable	Importancia
intensidad_lag_1h	0,846767
intensidad_lag_24h	0,073989
ocupacion_media	0,036419
hora	0,008396
velocidad_media	0,007674
media_movil_3h	0,007383

id	0,006790
media_movil_24h	0,004570
carga_media	0,004395
dia_semana	0,001736

Tabla 7. Variables más importantes del modelo Random Forest. Elaboración propia.

La variable más influyente es la intensidad de la hora anterior, lo cual resulta coherente con la naturaleza del tráfico urbano: el estado del tráfico en una franja horaria suele estar muy relacionado con el estado inmediatamente anterior. La segunda variable más relevante es la intensidad registrada en la misma hora del día anterior, lo que refleja la existencia de patrones diarios. La ocupación media y la velocidad media también aportan información complementaria sobre la situación operativa de la vía.

5.10. Diseño del cuadro de mando interactivo

La parte de visualización se ha implementado en Power BI a partir de los ficheros exportados por el script 03_export_powerbi.py. Para optimizar el rendimiento del dashboard se generaron tablas específicas para la visualización, incluyendo tablas agregadas por tiempo, por punto de medida, predicciones, métricas del modelo, importancia de variables, calidad de datos y resumen de limpieza. Esta separación evita cargar innecesariamente todos los datos detallados en cada visualización y facilita la construcción de páginas temáticas.

El dashboard se ha estructurado en cuatro páginas principales: calidad de datos, análisis temporal, análisis geográfico y modelo predictivo. Esta organización sigue el flujo lógico del proyecto: primero se documenta la calidad y el volumen del dataset, después se exploran los patrones temporales, a continuación, se analiza la distribución espacial de los puntos de medida y, finalmente, se presentan los resultados del modelo predictivo.

Estructura del dashboard desarrollado en Power BI:

- Calidad de datos → Resumen general, registros procesados, puntos analizados, nulos por columna y registros horarios generados por archivo mensual.

- Análisis temporal → Intensidad media por hora, día de la semana y mes, con segmentadores por año y mes.
- Análisis geográfico → Mapa o visualización espacial de los puntos de medida, ranking de puntos con mayor intensidad y tabla de detalle.
- Modelo predictivo → Comparación entre intensidad real y predicha, métricas MAE/RMSE/R², importancia de variables y error absoluto medio por hora.

5.11. Reutilización de la arquitectura en otros dominios

Aunque el caso práctico se centra en movilidad urbana, la arquitectura propuesta no está limitada exclusivamente al tráfico. Su diseño por capas permite adaptarse a otros dominios que compartan características similares: datos temporales, registros con referencias geográficas, necesidad de limpieza previa, variable objetivo predictiva y explotación mediante dashboard. Por ejemplo, la misma estructura podría aplicarse a datos de demanda energética, ocupación de aparcamientos, uso de transporte público, actividad turística o indicadores educativos.

Para reutilizar la arquitectura en otro dominio sería necesario sustituir la capa de adquisición de datos, adaptar las reglas de limpieza, redefinir la variable objetivo y modificar las visualizaciones del dashboard. Sin embargo, el patrón general se mantendría: ingesta de datos, transformación, construcción del dataset analítico, modelado predictivo, evaluación y visualización. Esta separación entre arquitectura general e instanciación concreta permite resolver la tensión entre el carácter general de la propuesta y el caso de movilidad utilizado para validarla.

6. Código fuente y datos analizados

Este apartado documenta la organización del código fuente desarrollado y los datos utilizados durante la implementación. La estructura del proyecto se diseñó para separar los datos originales, los datos procesados, los scripts, el dashboard, las figuras y la documentación. Esta separación facilita la trazabilidad del proceso y permite reproducir las fases principales de la arquitectura.

6.1. Organización del repositorio

El código fuente y los ficheros auxiliares del proyecto se organizaron en una carpeta principal denominada TFM_Laura. Dentro de ella se crearon subcarpetas específicas para el código, los datos, el dashboard, las figuras y la documentación. La estructura final utilizada fue la siguiente:

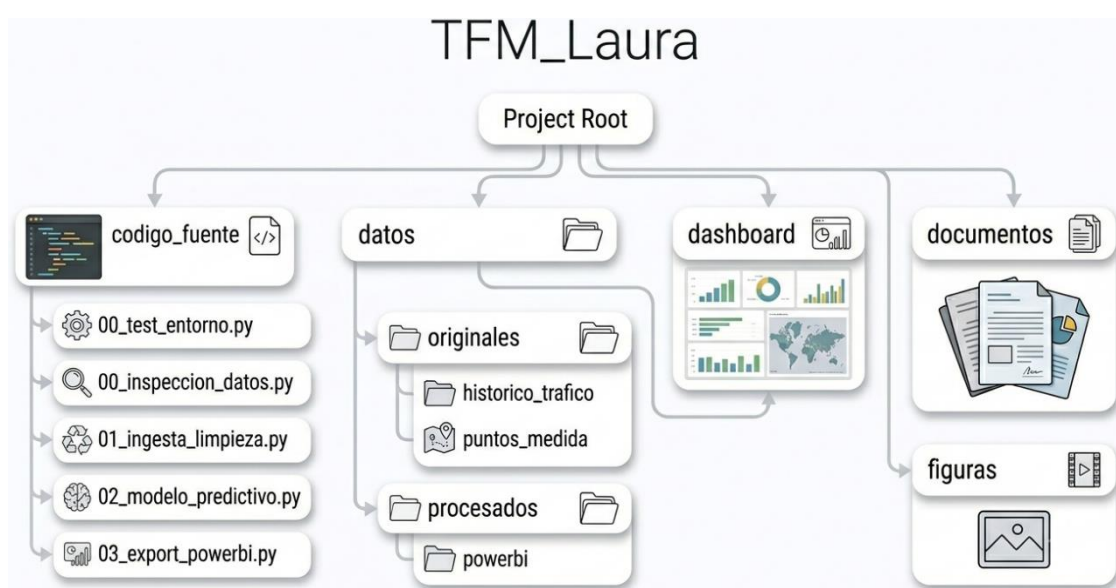


Figura 1. Organización del repositorio donde se encuentra el proyecto. Elaboración propia

6.2. Scripts desarrollados

Script	Función
00_test_entorno.py	Comprueba que el entorno virtual y las bibliotecas principales están correctamente instaladas.

00_inspeccion_datos.py	Inspecciona las columnas y primeras filas de los ficheros originales para adaptar el pipeline a la estructura real de los datos.
01_ingesta_limpieza.py	Lee los ficheros mensuales por bloques, filtra registros erróneos, agrega los datos a nivel horario y une la información geográfica.
02_modelo_predictivo.py	Genera variables históricas, divide el dataset en entrenamiento y prueba, entrena el modelo Random Forest y exporta métricas y predicciones.
03_export_powerbi.py	Prepara tablas específicas para Power BI, incluyendo datos temporales, puntos, predicciones, métricas, calidad de datos e importancia de variables.

Tabla 8. Scripts desarrollados durante el proyecto. Elaboración propia

6.3. Datos originales utilizados

Los datos originales se han almacenado en la carpeta datos/originales. En la subcarpeta historico_trafico se incluyen los 24 ficheros CSV correspondientes a los meses de enero a diciembre de 2023 y enero a diciembre de 2024. En la subcarpeta puntos_medida se incluye el fichero Excel de puntos de medida correspondiente a diciembre de 2024.

Carpeta	Contenido	Descripción
historico_trafico	24 ficheros CSV mensuales	Datos históricos de intensidad de tráfico de 2023 y 2024.
puntos_medida	1 fichero Excel	Ubicación de puntos de medida, incluyendo id, distrito, nombre, longitud y latitud.

Tabla 9. Datos originales utilizados. Elaboración propia.

6.4. Datos procesados generados

Archivo generado	Descripción
trafico_limpio.csv	Dataset horario final con 836.228 registros.
resumen_limpieza.csv	Resumen por fichero mensual con registros leídos, válidos y horarios generados.
puntos_medida_limpios.csv	Tabla depurada de puntos de medida.
puntos_seleccionados_modelo.csv	Lista de los 50 puntos seleccionados para el prototipo.
trafico_modelo.csv	Dataset con variables adicionales para modelado predictivo.
predicciones.csv	Valores reales, predicciones del baseline, predicciones del Random Forest y errores.
metricas_modelo.csv	Métricas MAE, RMSE y R^2 de los modelos evaluados.
importancia_variables.csv	Importancia de las variables según el modelo Random Forest.
powerbi/*.csv	Tablas optimizadas para la carga y visualización en Power BI.

Tabla 10. Archivos procesados generados por la arquitectura. Elaboración propia.

7. Conclusiones

7.1. Conclusiones del proyecto

El trabajo realizado me ha permitido diseñar, implementar y validar una arquitectura funcional de datos masivos orientada al análisis predictivo y a la visualización interactiva. La arquitectura se ha aplicado a un caso práctico de movilidad urbana basado en datos abiertos de tráfico de Madrid, demostrando que es posible integrar fuentes públicas, procesar grandes volúmenes de registros, construir un dataset analítico, entrenar modelos predictivos y comunicar los resultados a través de un dashboard interactivo.

Una de las principales aportaciones del proyecto es la construcción de un pipeline reproducible de ingesta y limpieza. A partir de 24 ficheros mensuales y 308.458.911 registros originales, se ha generado un dataset horario final de 836.228 registros correspondientes a 50 puntos de medida durante el tiempo que va entre enero de 2023 y diciembre de 2024. Esta transformación ha permitido pasar de datos brutos, repartidos en múltiples ficheros y con una granularidad inferior a la hora, a una estructura analítica clara y explotable.

Desde el punto de vista predictivo, el modelo Random Forest Regressor ha obtenido resultados claramente superiores al modelo base. La reducción del MAE de 433,41 a 164,16 vehículos/hora y el aumento del R^2 de 0,7569 a 0,9666 evidencian que las variables históricas, temporales y operativas utilizadas aportan información relevante para estimar la intensidad horaria del tráfico. Además, la importancia de variables muestra que la intensidad de la hora anterior y la intensidad de la misma hora del día anterior son los factores más determinantes, lo que resulta coherente con la naturaleza temporal y recurrente del tráfico urbano.

El dashboard desarrollado en Power BI completa la arquitectura al convertir los resultados técnicos en información visual e interactiva sencilla de interpretar. Las páginas de calidad de datos, análisis temporal, análisis geográfico y modelo predictivo permiten explorar el comportamiento del tráfico, identificar patrones horarios y semanales, localizar puntos de mayor intensidad y evaluar la precisión del modelo. De esta forma, la visualización actúa como una capa de comunicación y validación de la solución.

En conjunto, el TFM demuestra que una arquitectura modular basada en herramientas accesibles puede ofrecer una solución funcional para el análisis de datos temporales y

referenciados geográficamente. Aunque la implementación se ha realizado en entorno local y con una selección de puntos de medida, el diseño por capas facilita su ampliación futura hacia soluciones distribuidas, automatizadas o desplegadas en la nube.

7.2.Relación entre objetivos y resultados obtenidos

Descripción	Resultado obtenido
Analizar requisitos técnicos de una arquitectura de datos masivos.	Se revisaron las necesidades de ingesta, procesamiento, modelado y visualización.
Seleccionar e integrar fuentes abiertas de tráfico urbano.	Se utilizaron histórico de tráfico y puntos de medida del Ayuntamiento de Madrid.
Diseñar una arquitectura modular.	Se definieron capas de adquisición, ingesta, procesamiento, modelado y visualización.
Implementar limpieza y transformación reproducible.	Se desarrolló un pipeline Python que generó 836.228 registros horarios.
Desarrollar y evaluar modelos predictivos.	Se comparó baseline lag 24h con Random Forest, obteniendo $R^2=0,9666$.
Diseñar un dashboard interactivo.	Se creó un informe Power BI con cuatro páginas analíticas.
Valorar utilidad, limitaciones y reutilización.	Se documentaron limitaciones, trabajo futuro y posibles dominios de reutilización.

Tabla 11. Relación entre los objetivos específicos y los resultados obtenidos. Elaboración propia.

7.3.Conclusiones personales y aprendizaje adquirido

Desde un punto de vista personal, este trabajo me ha permitido consolidar conocimientos relacionados con el ciclo completo de un proyecto de datos: desde la descarga y organización

de fuentes abiertas hasta la construcción de un modelo predictivo y su visualización final. La implementación ha demostrado la importancia de dedicar tiempo a la preparación de los datos, ya que la calidad del modelo y del dashboard depende directamente de la calidad del dataset generado.

Al mismo tiempo, el desarrollo del proyecto me ha permitido profundizar en el uso de Python para automatizar procesos de limpieza y transformación, en la aplicación de modelos de aprendizaje automático con scikit-learn y en el diseño de cuadros de mando en Power BI. También he comprobado la necesidad de tomar decisiones técnicas realistas en función de los recursos disponibles. En este caso, el procesamiento por bloques me permitió trabajar con datos de gran tamaño en un entorno local, manteniendo la reproducibilidad y la trazabilidad de la solución.

Por último, el proyecto me ha mostrado la importancia de conectar la parte técnica con la comunicación de resultados. Un modelo predictivo no resulta útil únicamente por obtener buenas métricas, sino también por poder explicar sus resultados, interpretar sus variables más relevantes y presentarlos de forma clara a través de visualizaciones comprensibles

8. Limitaciones y prospectiva

8.1. Limitaciones

Aunque los resultados obtenidos son positivos, el trabajo presenta una serie de limitaciones que deben mostrarse. En primer lugar, la implementación se ha desarrollado en un entorno local y no en una infraestructura distribuida real. Por este motivo, aunque el pipeline trabaja con ficheros voluminosos mediante procesamiento por bloques, no puede considerarse equivalente a un despliegue Big Data completo basado en clústeres o servicios cloud.

En segundo lugar, para construir un prototipo funcional se han seleccionado los 50 puntos de medida con mayor número de registros válidos. Esta decisión permitió reducir el coste computacional y facilitar la validación de la arquitectura, pero implica que los resultados no representan la totalidad de la red de sensores de Madrid, por lo que los datos expuestos son limitados. En otros ámbitos se podría ampliar el análisis a un número mayor de puntos de medida o a la totalidad de los sensores disponibles.

En tercer lugar, el modelo predictivo se ha construido únicamente con variables procedentes del histórico de tráfico y de la ubicación de los puntos de medida. No se han incorporado variables externas como meteorología, calendario laboral, festivos, eventos, obras o incidencias. Estas variables podrían mejorar la capacidad predictiva del modelo, especialmente en situaciones poco frecuentes o comunes.

También debe señalarse que, en algunos meses de 2024, especialmente octubre y noviembre, se ha observado una reducción relevante en el número de registros válidos tras el filtrado. Esto puede estar asociado a la disponibilidad o calidad de los datos originales de la página y debe considerarse al interpretar los resultados. Por último, el dashboard desarrollado se plantea como un prototipo académico de TFM y no como una herramienta desplegada en producción.

8.2. Trabajo futuro

Como línea de trabajo futuro, la arquitectura podría ampliarse para procesar un periodo histórico más largo, incorporando todos los datos disponibles desde 2013. También sería recomendable incluir la totalidad de puntos de medida y no solo una selección de sensores, lo que permitiría construir una visión más completa del tráfico urbano de Madrid.

Otra mejora relevante consistiría en incorporar variables externas, como información meteorológica, días festivos, calendario laboral, eventos o incidencias de tráfico. Estas variables podrían ayudar a explicar desviaciones respecto a los patrones habituales y mejorar el rendimiento predictivo en situaciones excepcionales. También podrían probarse otros modelos de aprendizaje automático, como Gradient Boosting, XGBoost, redes neuronales recurrentes o modelos específicos de series temporales.

Desde el punto de vista de la arquitectura, una evolución natural sería migrar el procesamiento a una infraestructura distribuida o Cloud, utilizando tecnologías como Apache Spark, Dask, BigQuery, Databricks o servicios equivalentes. Esta evolución permitiría automatizar la ingesta periódica de nuevos datos, escalar el procesamiento y aproximar la solución a un entorno productivo.

Finalmente, el dashboard podría publicarse en Power BI Service, incorporando actualización automática de datos, control de acceso, alertas e informes compartidos. De este modo, la arquitectura pasaría de ser un prototipo académico a una solución analítica más cercana a un sistema de apoyo a la toma de decisiones.

Referencias bibliográficas

Tráfico. Histórico de datos del tráfico desde 2013 - Portal de datos abiertos del Ayuntamiento de Madrid. (s. f.). <https://datos.madrid.es/dataset/208627-0-transporte-ptomedida-historico>

Tráfico. Ubicación de los puntos de medida del tráfico - Portal de datos abiertos del Ayuntamiento de Madrid. (s. f.-b). <https://datos.madrid.es/dataset/202468-0-intensidad-trafico>

David, J. (2025, 14 julio). Top 10 clean code practices to reduce technical debt. Sparsity. <https://www.sparsity.com/blogs/the-importance-of-business-intelligence/>

Amazon Web Services. (s. f.). *¿Qué es extracción, transformación y carga (ETL)?* Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/etl/>

Ibm. (2026, 2 junio). Calidad de los datos. IBM. <https://www.ibm.com/es-es/think/topics/data-quality>

Capdevila, F. (2026, 22 abril). ➤ *¿Qué es Power BI? Guía completa: para qué sirve, cómo funciona y precios* (2026). Hako IT. <https://www.hakoit.com/que-es-power-bi/>

Cerveró, E. (2026, 21 enero). *¿Qué es Power BI, para qué sirve y cómo funciona?* Aglaia. <https://aglaia.es/blog/power-bi/que-es-power-bi-para-que-sirve-y-como-funciona/>

Estudio de la movilidad con Big Data | Ministerio de Transportes y Movilidad Sostenible. (s. f.). <https://www.transportes.gob.es/ministerio/proyectos-singulares/estudio-de-movilidad-con-big-data>

LA REVOLUCIÓN DEL BIG DATA EN EL TRANSPORTE: TRANSFORMANDO LA MOVILIDAD EN LA ERA DIGITAL. (s. f.). Master Big Data y Data Science UNED. <https://www.masterbigdataonline.com/blog/20250109-big-data-sector-transportes.html>

Martínez, S. (2026, 30 enero). Ciclo de vida de los datos: qué es, etapas y cómo gestionarlo correctamente. Inesdi. <https://www.inesdi.com/blog/ciclo-vida-datos/>

JulCsc. (s. f.). *Introducción a Power BI Desktop - Power BI.* Microsoft Learn. <https://learn.microsoft.com/es-es/power-bi/fundamentals/desktop-getting-started>

Ciclo de vida de los datos - AWS Orientación prescriptiva. (s. f.).

https://docs.aws.amazon.com/es_es/prescriptive-guidance/latest/modern-data-centric-use-cases/data-lifecycle.html

Hai, R., Koutras, C., Quix, C., & Jarke, M. (2023). Data Lakes: A Survey of Functions and Systems. IEEE Transactions On Knowledge And Data Engineering, 35(12), 12571-12590. <https://doi.org/10.1109/tkde.2023.3270101>

Bienvenido - EMT data. (s. f.). <https://datos.emtmadrid.es/>

Levin, W. L. C. D. B. O. (2020, 7 enero). NIST Big Data Interoperability Framework: Volume 6, Reference Architecture. NIST. <https://www.nist.gov/publications/nist-big-data-interoperability-framework-volume-6-reference-architecture>

JulCsc. (s. f.-b). Power BI documentation - Power BI. Microsoft Learn. <https://learn.microsoft.com/en-us/power-bi/>

Anexo A. Privacidad y protección de datos

Este anexo tiene como finalidad analizar si los datos utilizados en el Trabajo Fin de Máster implican tratamiento de datos personales y, en consecuencia, si resulta aplicable la normativa de protección de datos personales.

El TFM propone el diseño e implementación de una arquitectura general de datos masivos para el análisis predictivo y la visualización interactiva, validada mediante un caso práctico de movilidad urbana. Para ello, se prevé utilizar datos abiertos procedentes de fuentes institucionales, preferentemente en formato agregado, anonimizado o sin identificadores personales.

A.1. Tipo de datos tratados

Los datos previstos para la validación de la arquitectura estarán relacionados con movilidad urbana, como registros agregados de tráfico, puntos de medición, intensidad de circulación, franjas temporales, localización de sensores o información de transporte publicada en portales de datos abiertos.

No se prevé tratar datos personales identificados o identificables, como nombres, DNI, matrículas, identificadores de dispositivos móviles, trayectorias individuales o datos que permitan reconstruir el comportamiento de una persona concreta.

La Agencia Española de Protección de Datos indica que el RGPD no afecta al tratamiento de información anónima cuando esta no guarda relación con una persona física identificada o identificable, o cuando los datos se han anonimizado de forma que la persona no pueda ser identificada. También señala que debe valorarse si existe una probabilidad razonable de identificación.

A.2. Aplicación o exclusión de la normativa de protección de datos

En función del tipo de datos previstos, este TFM se acoge al procedimiento de exclusión de la normativa de protección de datos personales, siempre que los conjuntos de datos finalmente

utilizados estén agregados, anonimizados o no permitan identificar directa ni indirectamente a personas físicas.

No obstante, se tendrá en cuenta que la AEPD considera datos personales cualquier información relativa a una persona física identificada o identificable, incluyendo datos de localización o identificadores en línea cuando permitan identificar a una persona. Por este motivo, antes de utilizar cada conjunto de datos se revisará si contiene información personal.

Si durante el desarrollo del trabajo se detectase que algún conjunto de datos pudiera permitir la identificación de personas, se descartará su uso o se sustituirá por una fuente agregada o anonimizada. En caso de no poder sustituirse, sería necesario aplicar el RGPD y la Ley Orgánica 3/2018, que adapta el ordenamiento jurídico español al Reglamento (UE) 2016/679 y se aplica al tratamiento automatizado o no automatizado de datos personales.

A.3. Finalidad del tratamiento

La finalidad del tratamiento de los datos será exclusivamente académica. Los datos se utilizarán para validar el funcionamiento de la arquitectura propuesta, comprobar su capacidad para integrar y procesar información de movilidad urbana, aplicar técnicas de análisis predictivo y representar los resultados mediante visualizaciones interactivas.

No se utilizarán los datos para tomar decisiones automatizadas con efectos sobre personas, elaborar perfiles individuales, realizar seguimiento de usuarios ni identificar patrones de comportamiento personal.

A.4. Medidas aplicadas

Durante el desarrollo del TFM se aplicarán las siguientes medidas:

- Utilizar únicamente datos abiertos, agregados o anonimizados.
- Evitar el uso de identificadores personales o información individualizada.
- No combinar fuentes con el objetivo de reconstruir trayectorias individuales.
- Documentar el origen de los datos utilizados.
- Citar correctamente las fuentes de información.

- Mantener una copia de los datos originales y otra de los datos transformados.
- Utilizar los datos únicamente con finalidad académica.
- Eliminar variables que no sean necesarias para los objetivos del análisis.

Estas medidas siguen un enfoque preventivo basado en la minimización del riesgo y en el uso responsable de la información.

A.5. Reutilización de datos abiertos

Al tratarse de datos procedentes previsiblemente de portales institucionales, también se respetará la normativa sobre reutilización de información del sector público. La Ley 37/2007 regula la reutilización de documentos elaborados o custodiados por administraciones y organismos del sector público.

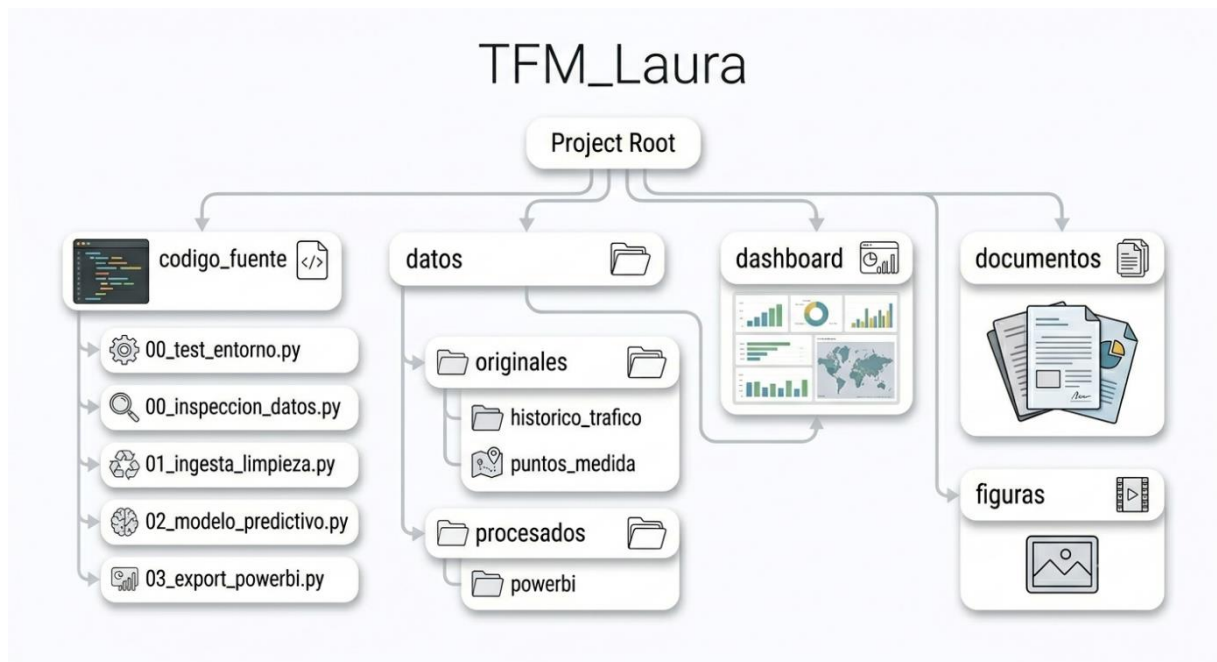
En particular, se tendrán en cuenta las condiciones generales de reutilización, como citar la fuente, no desnaturalizar el sentido de la información, mencionar la fecha de actualización cuando esté disponible y no revertir procedimientos de disociación mediante la combinación con otras fuentes.

A.6. Conclusión

En conclusión, el TFM no prevé el tratamiento de datos personales identificados o identificables. La validación de la arquitectura se realizará preferentemente con datos abiertos, agregados o anonimizados, por lo que el tratamiento principal quedaría fuera del ámbito de aplicación del RGPD.

Aun así, se aplicarán buenas prácticas de privacidad, minimización, documentación y reutilización responsable de la información. Si durante el desarrollo se identificase algún conjunto de datos con riesgo, se descartará o se sustituirá por una fuente anonimizada. De esta forma, el trabajo garantiza un tratamiento prudente y responsable de los datos utilizados.

Anexo B. Estructura de carpetas del proyecto



Anexo C. Script de ingesta y limpieza

```

from pathlib import Path
import pandas as pd

# =====
# CONFIGURACIÓN GENERAL
# =====

BASE = Path(".")
RUTA_HISTORICO = BASE / "datos" / "originales" / "historico_trafico"
RUTA_PUNTOS = BASE / "datos" / "originales" / "puntos_medida"
RUTA_PROCESADOS = BASE / "datos" / "procesados"

RUTA_PROCESADOS.mkdir(parents=True, exist_ok=True)

COLUMNAS_HISTORICO = [
    "id",
    "fecha",
    "tipo_elem",
    "intensidad",
    "ocupacion",
    "carga",
    "vmed",
    "error",
    "periodo_integracion"

```

]

CHUNKSIZE = 500_000

MAX_PUNTOS = 50

=====

FUNCIONES AUXILIARES

=====

def leer_puntos_medida():

"""

 Lee el archivo Excel de puntos de medida y devuelve una tabla limpia
 con id, nombre, distrito, longitud y latitud.

"""

archivos = list(RUTA_PUNTOS.glob("*.xlsx")) + list(RUTA_PUNTOS.glob("*.xls"))

if not archivos:

 raise FileNotFoundError("No se ha encontrado ningún archivo Excel en
datos/originales/puntos_medida")

archivo = archivos[0]

print(f"Leyendo puntos de medida: {archivo.name}")

puntos = pd.read_excel(archivo)

```
puntos.columns = (
    puntos.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
```

columnas_necesarias = ["id", "tipo_elem", "distrito", "nombre", "longitud", "latitud"]

columnas_existentes = [c for c in columnas_necesarias if c in puntos.columns]

puntos = puntos[columnas_existentes].copy()

puntos = puntos.drop_duplicates(subset=["id"])

puntos["id"] = pd.to_numeric(puntos["id"], errors="coerce")

puntos = puntos.dropna(subset=["id"])

puntos["id"] = puntos["id"].astype(int)

if "longitud" in puntos.columns:

puntos["longitud"] = pd.to_numeric(puntos["longitud"], errors="coerce")

if "latitud" in puntos.columns:

```

puntos["latitud"] = pd.to_numeric(puntos["latitud"], errors="coerce")

puntos.to_csv(RUTA_PROCESADOS / "puntos_medida_limpios.csv", index=False, sep=";")

print(f"Puntos de medida limpios: {len(puntos)}")
return puntos

def seleccionar_top_puntos(archivo_referencia):
    """
    Para que el proceso sea rápido y viable para la entrega,
    selecciona los puntos con más registros válidos en el primer archivo.
    """
    print(f"Seleccionando los {MAX_PUNTOS} puntos con más registros usando:
    {archivo_referencia.name}")

    conteos = {}

    for chunk in pd.read_csv(
        archivo_referencia,
        sep=";",
        encoding="utf-8",
        usecols=["id", "error"],
        chunksize=CHUNKSIZE
    ):
        chunk = chunk[chunk["error"] == "N"]
        vc = chunk["id"].value_counts()

        for id_punto, n in vc.items():
            conteos[id_punto] = conteos.get(id_punto, 0) + n

    serie = pd.Series(conteos).sort_values(ascending=False)
    top_ids = serie.head(MAX_PUNTOS).index.astype(int).tolist()

    print(f"Puntos seleccionados: {len(top_ids)}")
    print(f"Primeros puntos seleccionados: {top_ids[:10]}")

    pd.DataFrame({
        "id": top_ids,
        "registros_archivo_referencia": serie.head(MAX_PUNTOS).values
    }).to_csv(RUTA_PROCESADOS / "puntos_seleccionados_modelo.csv", index=False, sep=";")

    return top_ids

def procesar_archivo_historico(archivo, ids_seleccionados):
    """

```

Procesa un CSV mensual por bloques:

- filtra errores
- convierte fechas
- convierte numéricos
- filtra intensidades negativas
- agrega por hora y punto de medida

```
"""
print(f"\nProcesando archivo: {archivo.name}")

grupos_archivo = []
registros_leidos = 0
registros_validos = 0

for chunk in pd.read_csv(
    archivo,
    sep=";",
    encoding="utf-8",
    usecols=COLUMNAS_HISTORICO,
    chunksize=CHUNKSIZE
):
    registros_leidos += len(chunk)

    # Seleccionar solo puntos elegidos
    chunk = chunk[chunk["id"].isin(ids_seleccionados)]

    # Filtrar registros sin error
    chunk = chunk[chunk["error"] == "N"]

    # Convertir fecha
    chunk["fecha"] = pd.to_datetime(chunk["fecha"], errors="coerce")
    chunk = chunk.dropna(subset=["fecha"])

    # Convertir numéricos
    for col in ["intensidad", "ocupacion", "carga", "vmed", "periodo_integracion"]:
        chunk[col] = pd.to_numeric(chunk[col], errors="coerce")

    # Eliminar intensidad nula o negativa
    chunk = chunk.dropna(subset=["intensidad"])
    chunk = chunk[chunk["intensidad"] >= 0]

    registros_validos += len(chunk)

    # Agregación horaria
    chunk["fecha_hora"] = chunk["fecha"].dt.floor("h")

    grupo = (
        chunk
```

```

.groupby(["id", "fecha_hora"], as_index=False)
.agg(
    intensidad_horaria=("intensidad", "sum"),
    ocupacion_media=("ocupacion", "mean"),
    carga_media=("carga", "mean"),
    velocidad_media=("vmed", "mean"),
    periodo_integracion_medio=("periodo_integracion", "mean"),
    registros_15min=("intensidad", "count")
)
)

grupos_archivo.append(grupo)

if grupos_archivo:
    resultado = pd.concat(grupos_archivo, ignore_index=True)

# Reagrupar por si el mismo id/hora aparece en distintos chunks
resultado = (
    resultado
    .groupby(["id", "fecha_hora"], as_index=False)
    .agg(
        intensidad_horaria=("intensidad_horaria", "sum"),
        ocupacion_media=("ocupacion_media", "mean"),
        carga_media=("carga_media", "mean"),
        velocidad_media=("velocidad_media", "mean"),
        periodo_integracion_medio=("periodo_integracion_medio", "mean"),
        registros_15min=("registros_15min", "sum")
    )
)
else:
    resultado = pd.DataFrame()

print(f"Registros leídos: {registros_leidos}")
print(f"Registros válidos tras limpieza: {registros_validos}")
print(f"Registros horarios generados: {len(resultado)}")

resumen = {
    "archivo": archivo.name,
    "registros_leidos": registros_leidos,
    "registros_validos": registros_validos,
    "registros_horarios": len(resultado)
}

return resultado, resumen

# =====

```

PROCESO PRINCIPAL

=====

```

def main():
    print("=" * 100)
    print("INICIO DEL PROCESO DE INGESTA Y LIMPIEZA")
    print("=" * 100)

    archivos_historico = sorted(RUTA_HISTORICO.glob("*.csv"))

    if not archivos_historico:
        raise FileNotFoundError("No se han encontrado CSV en
datos/originales/historico_trafico")

    print(f"Archivos históricos encontrados: {len(archivos_historico)}")

    puntos = leer_puntos_medida()

    ids_seleccionados = seleccionar_top_puntos(archivos_historico[0])

    resultados = []
    resúmenes = []

    for archivo in archivos_historico:
        df_mes, resumen = procesar_archivo_historico(archivo, ids_seleccionados)

        if not df_mes.empty:
            resultados.append(df_mes)

            resúmenes.append(resumen)

    print("\nUniendo todos los meses procesados...")

    trafico = pd.concat(resultados, ignore_index=True)

    # Crear variables temporales
    trafico["año"] = trafico["fecha_hora"].dt.year
    trafico["mes"] = trafico["fecha_hora"].dt.month
    trafico["dia"] = trafico["fecha_hora"].dt.day
    trafico["hora"] = trafico["fecha_hora"].dt.hour
    trafico["dia_semana"] = trafico["fecha_hora"].dt.dayofweek
    trafico["es_fin_semana"] = trafico["dia_semana"].isin([5, 6]).astype(int)

    # Unir coordenadas
    trafico = trafico.merge(
        puntos,
        on="id",

```

```

    how="left"
)

# Ordenar
trafico = trafico.sort_values(["id", "fecha_hora"])

# Exportar dataset limpio
salida_trafico = RUTA_PROCESADOS / "trafico_limpio.csv"
trafico.to_csv(salida_trafico, index=False, sep=";")

# Exportar resumen de limpieza
resumen_limpieza = pd.DataFrame(resumenes)
resumen_limpieza.to_csv(RUTA_PROCESADOS / "resumen_limpieza.csv", index=False,
sep=";")

print("\n" + "=" * 100)
print("PROCESO FINALIZADO")
print("=" * 100)
print(f"Archivo generado: {salida_trafico}")
print(f"Registros finales: {len(trafico)}")
print(f"Puntos de medida finales: {trafico['id'].nunique()}")
print(f"Periodo inicial: {trafico['fecha_hora'].min()}")
print(f"Periodo final: {trafico['fecha_hora'].max()}")
print("También se ha generado: datos/procesados/resumen_limpieza.csv")
print("También se ha generado: datos/procesados/puntos_medida_limpios.csv")
print("También se ha generado: datos/procesados/puntos_seleccionados_modelo.csv")

if __name__ == "__main__":
    main()

```

Anexo D. Script de modelado predictivo

```

from pathlib import Path
import pandas as pd
import numpy as np

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# =====
# CONFIGURACIÓN
# =====

BASE = Path(".")
RUTA_PROCESADOS = BASE / "datos" / "procesados"

```

```

ARCHIVO_ENTRADA = RUTA_PROCESADOS / "trafico_limpio.csv"
ARCHIVO_MODELO = RUTA_PROCESADOS / "trafico_modelo.csv"
ARCHIVO_PREDICCIONES = RUTA_PROCESADOS / "predicciones.csv"
ARCHIVO_METRICAS = RUTA_PROCESADOS / "metricas_modelo.csv"

# =====
# CARGA DE DATOS
# =====

print("=" * 100)
print("INICIO DEL MODELO PREDICTIVO")
print("=" * 100)

print(f"Leyendo dataset limpio: {ARCHIVO_ENTRADA}")
df = pd.read_csv(ARCHIVO_ENTRADA, sep=";")

df["fecha_hora"] = pd.to_datetime(df["fecha_hora"], errors="coerce")

df = df.dropna(subset=["fecha_hora", "id", "intensidad_horaria"])
df = df.sort_values(["id", "fecha_hora"])

print(f"Registros leídos: {len(df)}")
print(f"Puntos de medida: {df['id'].nunique()}")
print(f"Periodo inicial: {df['fecha_hora'].min()}")
print(f"Periodo final: {df['fecha_hora'].max()}")

# =====
# CREACIÓN DE VARIABLES
# =====

print("\nCreando variables predictivas...")

df["intensidad_lag_1h"] = df.groupby("id")["intensidad_horaria"].shift(1)
df["intensidad_lag_24h"] = df.groupby("id")["intensidad_horaria"].shift(24)

df["media_movil_3h"] = (
    df.groupby("id")["intensidad_horaria"]
    .shift(1)
    .rolling(window=3)
    .mean()
    .reset_index(level=0, drop=True)
)

df["media_movil_24h"] = (
    df.groupby("id")["intensidad_horaria"]
    .shift(1)

```

```

        .rolling(window=24)
        .mean()
        .reset_index(level=0, drop=True)
    )

    # Eliminar filas sin variables históricas
    df_modelo = df.dropna(subset=[
        "intensidad_lag_1h",
        "intensidad_lag_24h",
        "media_movil_3h",
        "media_movil_24h"
    ]).copy()

    print(f"Registros disponibles para modelo: {len(df_modelo)}")

    # =====
    # VARIABLES DEL MODELO
    # =====

    features = [
        "id",
        "año",
        "mes",
        "dia",
        "hora",
        "dia_semana",
        "es_fin_semana",
        "ocupacion_medio",
        "carga_medio",
        "velocidad_medio",
        "registros_15min",
        "intensidad_lag_1h",
        "intensidad_lag_24h",
        "media_movil_3h",
        "media_movil_24h"
    ]

    # Nos quedamos solo con las columnas que existan
    features = [col for col in features if col in df_modelo.columns]

    target = "intensidad_horaria"

    df_modelo = df_modelo.dropna(subset=features + [target])

    # =====
    # DIVISIÓN TEMPORAL
    # =====

```

```

# Entrenamiento: año 2023
# Prueba: año 2024
train = df_modelo[df_modelo["año"] == 2023].copy()
test = df_modelo[df_modelo["año"] == 2024].copy()

print(f"Registros entrenamiento 2023: {len(train)}")
print(f"Registros prueba 2024: {len(test)}")

X_train = train[features]
y_train = train[target]

X_test = test[features]
y_test = test[target]

# =====
# MODELO BASE
# =====

print("\nEvaluando modelo base...")

# Modelo base: predice la intensidad de la misma hora del día anterior
baseline_pred = test["intensidad_lag_24h"]

mae_base = mean_absolute_error(y_test, baseline_pred)
rmse_base = np.sqrt(mean_squared_error(y_test, baseline_pred))
r2_base = r2_score(y_test, baseline_pred)

# =====
# MODELO RANDOM FOREST
# =====

print("Entrenando Random Forest...")

modelo_rf = RandomForestRegressor(
    n_estimators=100,
    max_depth=14,
    min_samples_leaf=5,
    random_state=42,
    n_jobs=-1
)

modelo_rf.fit(X_train, y_train)

pred_rf = modelo_rf.predict(X_test)

mae_rf = mean_absolute_error(y_test, pred_rf)

```

```

rmse_rf = np.sqrt(mean_squared_error(y_test, pred_rf))
r2_rf = r2_score(y_test, pred_rf)

# =====
# EXPORTAR MÉTRICAS
# =====

metricas = pd.DataFrame([
    {
        "modelo": "Baseline lag 24h",
        "MAE": mae_base,
        "RMSE": rmse_base,
        "R2": r2_base
    },
    {
        "modelo": "Random Forest Regressor",
        "MAE": mae_rf,
        "RMSE": rmse_rf,
        "R2": r2_rf
    }
])

metricas.to_csv(ARCHIVO_METRICAS, index=False, sep=";")

print("\nMétricas del modelo:")
print(metricas)

# =====
# EXPORTAR PREDICCIONES
# =====

predicciones = test.copy()

predicciones["prediccion_baseline"] = baseline_pred.values
predicciones["prediccion_rf"] = pred_rf
predicciones["error_rf"] = predicciones["intensidad_horaria"] - predicciones["prediccion_rf"]
predicciones["error_abs_rf"] = predicciones["error_rf"].abs()

# Para Power BI y memoria, no necesitamos demasiadas columnas
columnas_predicciones = [
    "id",
    "fecha_hora",
    "año",
    "mes",
    "dia",
    "hora",
    "dia_semana",

```

```

"es_fin_semana",
"intensidad_horaria",
"prediccion_baseline",
"prediccion_rf",
"error_rf",
"error_abs_rf",
"latitud",
"longitud",
"nombre",
"distrito"
]

columnas_predicciones = [c for c in columnas_predicciones if c in predicciones.columns]

predicciones[columnas_predicciones].to_csv(
    ARCHIVO_PREDICCIONES,
    index=False,
    sep=";"
)

# Exportar dataset usado para el modelo
df_modelo.to_csv(ARCHIVO_MODELO, index=False, sep=";")

# =====
# IMPORTANCIA DE VARIABLES
# =====

importancias = pd.DataFrame({
    "variable": features,
    "importancia": modelo_rf.feature_importances_
}).sort_values("importancia", ascending=False)

importancias.to_csv(
    RUTA_PROCESADOS / "importancia_variables.csv",
    index=False,
    sep=";"
)

print("\nVariables más importantes:")
print(importancias.head(10))

print("\n" + "=" * 100)
print("MODELO FINALIZADO")
print("=" * 100)
print(f"Archivo generado: {ARCHIVO_MODELO}")
print(f"Archivo generado: {ARCHIVO_PREDICCIONES}")
print(f"Archivo generado: {ARCHIVO_METRICAS}")

```

```
print(f"Archivo generado: {RUTA_PROCESADOS / 'importancia_variables.csv'})")
```

Anexo E. Script de exportación a Power BI

```
from pathlib import Path
import pandas as pd

# =====
# CONFIGURACIÓN
# =====

BASE = Path(".")
RUTA_PROCESADOS = BASE / "datos" / "procesados"
RUTA_POWERBI = RUTA_PROCESADOS / "powerbi"

RUTA_POWERBI.mkdir(parents=True, exist_ok=True)

# =====
# CARGA DE ARCHIVOS
# =====

print("=" * 100)
print("PREPARANDO ARCHIVOS PARA POWER BI")
print("=" * 100)

trafico = pd.read_csv(RUTA_PROCESADOS / "trafico_limpio.csv", sep=";")
predicciones = pd.read_csv(RUTA_PROCESADOS / "predicciones.csv", sep=";")
metricas = pd.read_csv(RUTA_PROCESADOS / "metricas_modelo.csv", sep=";")
importancias = pd.read_csv(RUTA_PROCESADOS / "importancia_variables.csv", sep=";")
resumen_limpieza = pd.read_csv(RUTA_PROCESADOS / "resumen_limpieza.csv", sep=";")
puntos = pd.read_csv(RUTA_PROCESADOS / "puntos_medida_limpios.csv", sep=";")

# =====
# PREPARACIÓN DE FECHAS
# =====

trafico["fecha_hora"] = pd.to_datetime(trafico["fecha_hora"], errors="coerce")
predicciones["fecha_hora"] = pd.to_datetime(predicciones["fecha_hora"], errors="coerce")

trafico["fecha"] = trafico["fecha_hora"].dt.date
predicciones["fecha"] = predicciones["fecha_hora"].dt.date

# =====
# TABLA RESUMEN GENERAL
# =====
```

```

resumen_general = pd.DataFrame([
    {
        "indicador": "Registros dataset limpio",
        "valor": len(trafico)
    },
    {
        "indicador": "Puntos de medida analizados",
        "valor": trafico["id"].nunique()
    },
    {
        "indicador": "Fecha inicial",
        "valor": str(trafico["fecha_hora"].min())
    },
    {
        "indicador": "Fecha final",
        "valor": str(trafico["fecha_hora"].max())
    },
    {
        "indicador": "Registros de entrenamiento 2023",
        "valor": len(trafico[trafico["año"] == 2023])
    },
    {
        "indicador": "Registros de prueba 2024",
        "valor": len(trafico[trafico["año"] == 2024])
    }
])

# =====
# TABLA DE CALIDAD DE DATOS
# =====

calidad_datos = pd.DataFrame({
    "columna": trafico.columns,
    "nulos": trafico.isna().sum().values,
    "porcentaje_nulos": (trafico.isna().sum().values / len(trafico)) * 100
})

# =====
# TABLA TEMPORAL AGREGADA
# =====

trafico_temporal = (
    trafico
    .groupby(["año", "mes", "dia_semana", "hora"], as_index=False)
    .agg(
        intensidad_media=("intensidad_horaria", "mean"),

```

```

intensidad_total=("intensidad_horaria", "sum"),
ocupacion_media=("ocupacion_media", "mean"),
velocidad_media=("velocidad_media", "mean"),
registros=("id", "count")
    )
)

# =====
# TABLA POR PUNTO DE MEDIDA
# =====

trafico_puntos = (
    trafico
    .groupby(["id", "nombre", "distrito", "latitud", "longitud"], as_index=False)
    .agg(
        intensidad_media=("intensidad_horaria", "mean"),
        intensidad_total=("intensidad_horaria", "sum"),
        ocupacion_media=("ocupacion_media", "mean"),
        velocidad_media=("velocidad_media", "mean"),
        registros=("fecha_hora", "count")
    )
)

# =====
# TABLA DE PREDICCIONES REDUCIDA
# =====

# Para Power BI no hace falta cargar absolutamente todas las columnas.
columnas_pred = [
    "id",
    "fecha_hora",
    "fecha",
    "año",
    "mes",
    "dia",
    "hora",
    "dia_semana",
    "es_fin_semana",
    "intensidad_horaria",
    "prediccion_baseline",
    "prediccion_rf",
    "error_rf",
    "error_abs_rf",
    "latitud",
    "longitud",
    "nombre",
    "distrito"

```

]

```
columnas_pred = [c for c in columnas_pred if c in predicciones.columns]
predicciones_powerbi = predicciones[columnas_pred].copy()
```

```
# =====
# EXPORTACIÓN
# =====
```

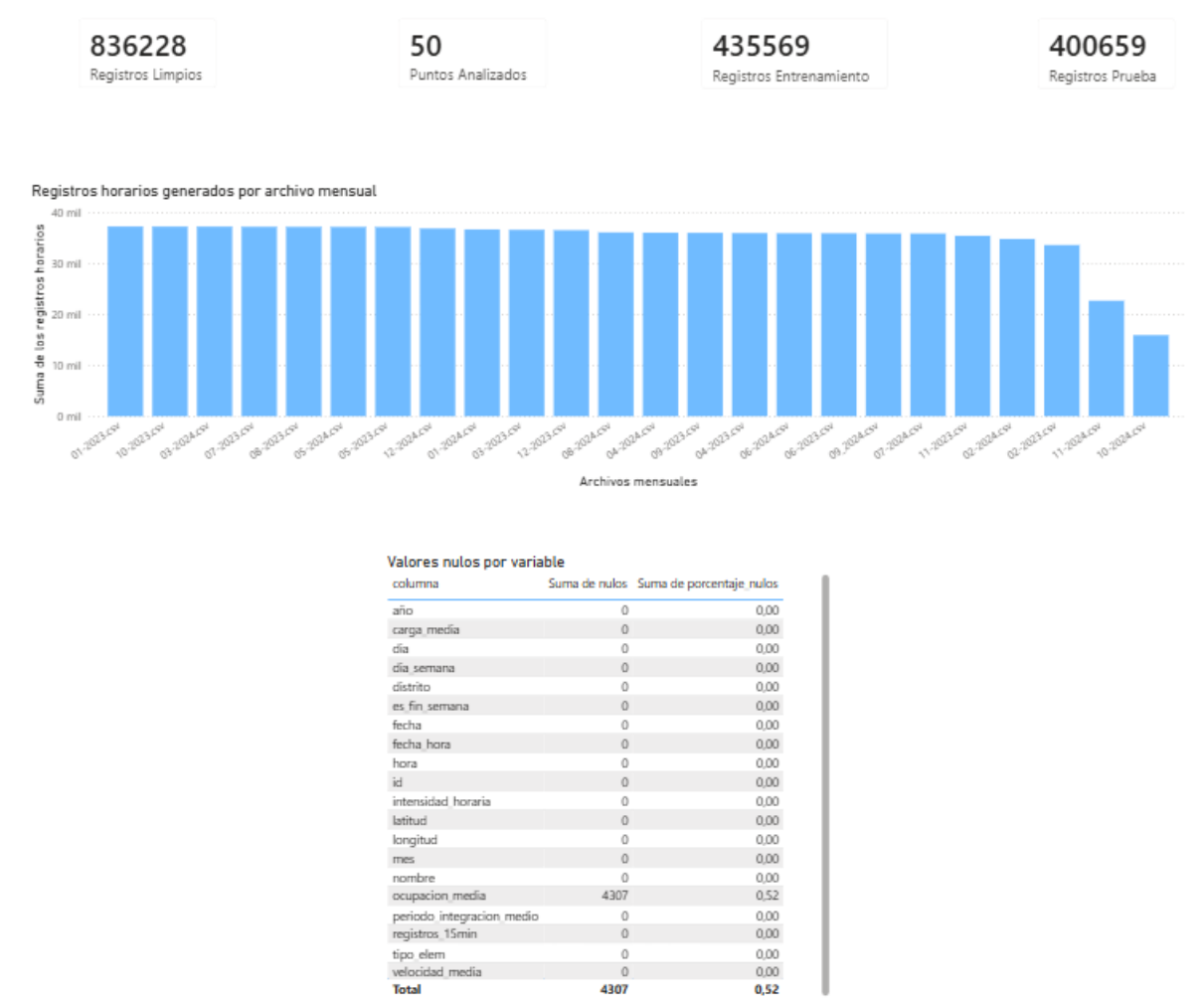
```
trafico.to_csv(RUTA_POWERBI / "trafico_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
trafico_temporal.to_csv(RUTA_POWERBI / "trafico_temporal_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
trafico_puntos.to_csv(RUTA_POWERBI / "trafico_puntos_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
predicciones_powerbi.to_csv(RUTA_POWERBI / "predicciones_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
metricas.to_csv(RUTA_POWERBI / "metricas_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
importancias.to_csv(RUTA_POWERBI / "importancia_variables_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
resumen_limpieza.to_csv(RUTA_POWERBI / "resumen_limpieza_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
resumen_general.to_csv(RUTA_POWERBI / "resumen_general_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
calidad_datos.to_csv(RUTA_POWERBI / "calidad_datos_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
puntos.to_csv(RUTA_POWERBI / "puntos_medida_powerbi.csv", index=False, sep=";", encoding="utf-8-sig", decimal=",")
```

```
print("Archivos generados en datos/procesados/powerbi:")
for archivo in RUTA_POWERBI.glob("*.csv"):
    print(f"- {archivo.name}")
```

```
print("\nProceso finalizado correctamente.")
```

Anexo F. Imágenes del Dashboard en Power BI

Página 1 – Calidad de datos

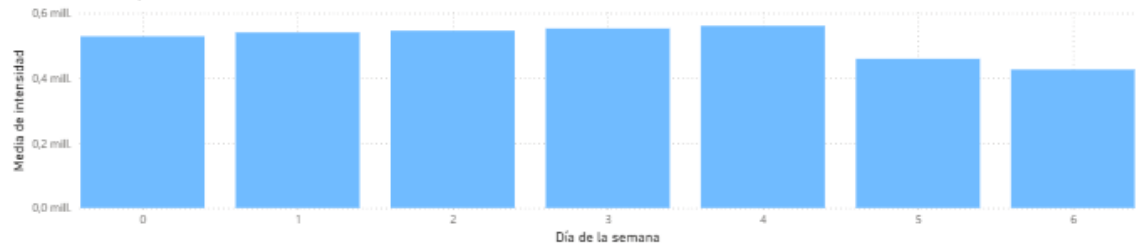


Página 2 – Análisis temporal

Intensidad media por hora del día

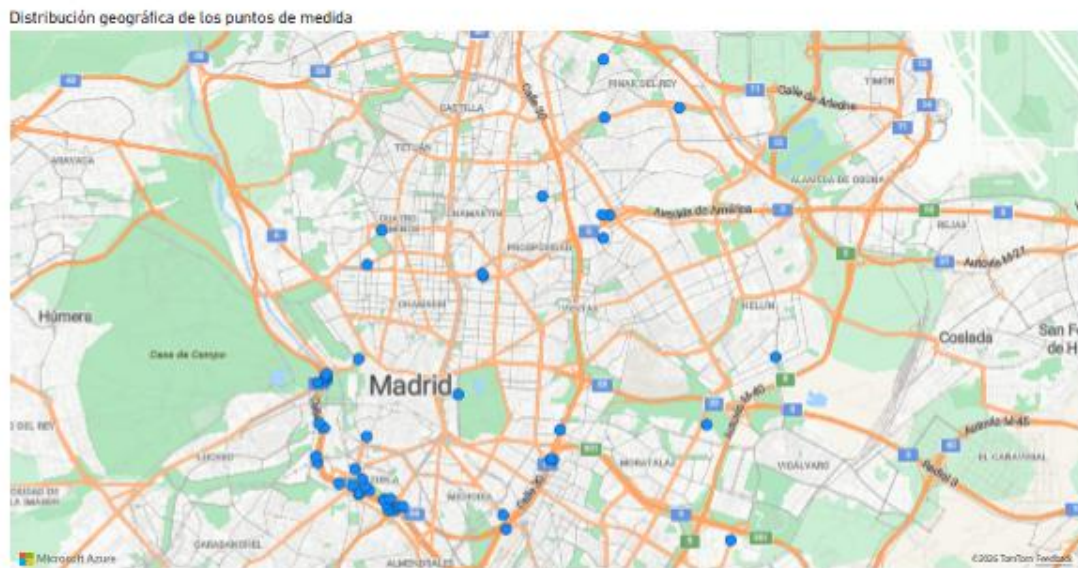


Intensidad media por día de la semana



Evolución mensual de la intensidad media





Puntos con mayor intensidad media

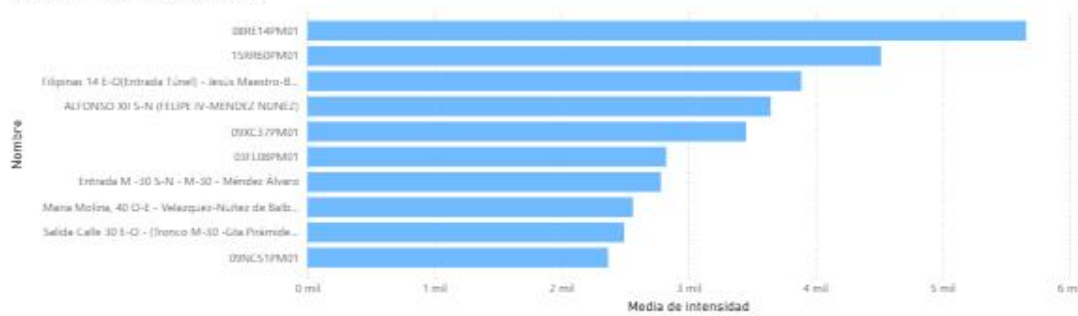
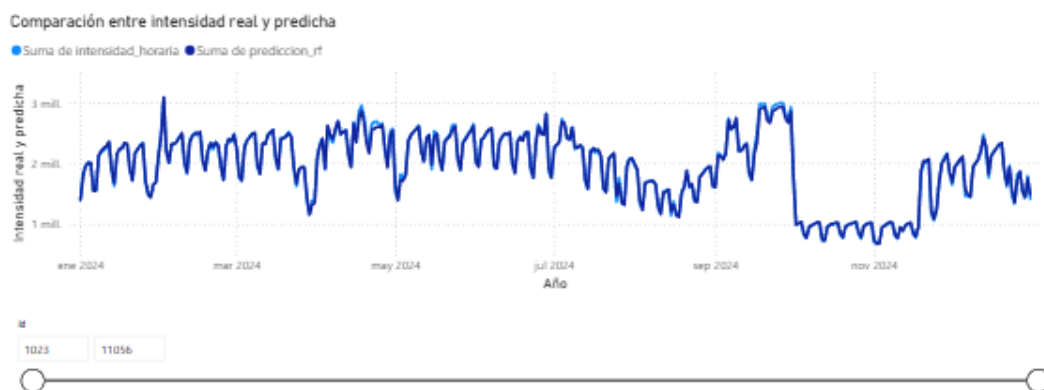


Tabla detalle

id	nombre	distrito	intensidad_medio	velocidad_medio	registros
1023	14RN01PM01	11	2.283,04	36,52	16276
1024	14RL30PM01	12	503,79	34,54	16368
1025	14RA15PM01	2	554,21	45,01	16368
1026	14RM32PM01	2	945,79	44,63	16368
1027	14VED4PM01	2	2.223,10	36,41	16360
1028	14RC10PM01	2	2.001,73	32,71	16368
1029	14RH25PM01	2	1.394,20	40,57	16368
1030	14RE35PM01	11	1.519,02	41,19	16352
1031	15RR60PM01	2	4.515,07	30,14	16360
1032	15RC05PM01	2	1.035,25	44,07	16360
1033	15RH53PM01	11	1.379,78	45,81	16360
1034	15RN03PM01	11	1.066,55	42,60	16360
1035	15RL11PM01	11	995,09	44,17	16368
1036	15RT07PM01	11	1.561,77	43,97	16363
1037	15RR22PM01	11	2.042,83	38,71	16363
1038	17RA10PM01	2	1.965,27	51,62	16360

Página 4 – Modelo predictivo



164,16

MAE Random Forest

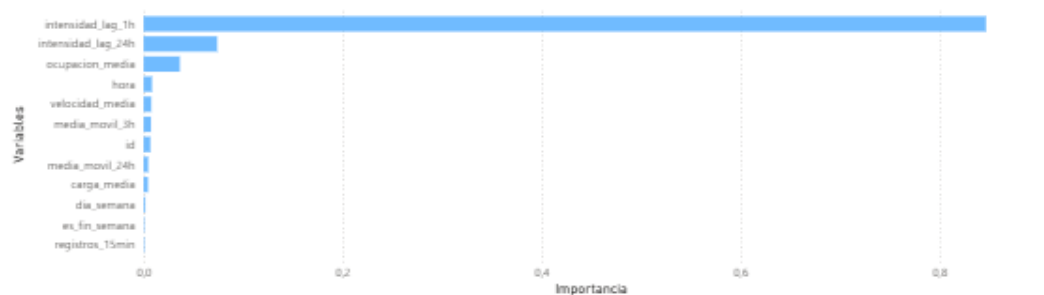
311,83

RMSE Random Forest

0,97

R2 Random Forest

Importancia de variables del modelo Random Forest



Error absoluto medio por hora

